BELIEF UPDATING AT FIXED DEPTH

Working draft

Introduction

How can a transformer with a fixed number of layers perform Bayesian inference over a long sequence of observations? In a hidden Markov model (HMM), the *belief* is the posterior distribution of the current hidden state given the observed history. Besides their strikingly cute geometry, beliefs are sufficient for prediction: the conditional distribution of every future sequence depends on the history only through the belief. That gives us something precise to look for inside a trained predictor. Linear probes have found these beliefs in transformers trained on HMM-generated data, fractal geometry and all [1].

Finding the beliefs still leaves the awkward question of how they got there. Bayes rule gives us humans a recipe: observe, update, repeat. A transformer, inconveniently, does not grow another layer every time we feed it another token. To reproduce a long sequence of updates with a few layers, it must reorganize the computation. This is where the *shortcuts* come in, and where the algebra has something to say.

When the belief set is finite, we can apply the classical algebraic decompositions used by Liu et al. [2], which give constant-depth shortcuts under a solvability condition. A Markov model with finitely many hidden states, however, has made no promise to keep its beliefs finite, and even the most optimistic observer would struggle at finding cuteness when staring at finitely many points. We therefore seek an algebraic decomposition of belief updates that still works when the resulting transformations form an infinite set.

We work with generalized hidden Markov models (GHMMs), which express word probabilities through products of real matrices. They include ordinary HMMs but permit negative matrix entries, provided the resulting word probabilities remain valid. Singular matrices are allowed as well.

Our first result applies to *any* monoid of real matrices: its products can be recovered from a finite cascade built from group actions, *resets* and finite transformation monoids (Theorem 2.7). The theory also builds the group factors from generally non-invertible matrices. In the cases of interest for the beauty-seaker, these groups will be infinite. They are generated by restrictions of monoid elements acting invertibly on *eventual images*: images of matrix products that map their own image onto itself. This separates changes of rank from multiplication within these groups.

Our main theorem shows that *solvability of these groups is enough for constant-depth belief updating* (Theorem 3.1). This holds for every GHMM in the class, including those with infinitely many beliefs and arbitrary real parameters. Algebraic group theory supplies the structure needed to compute with the infinite factors in parallel up to arbitrary error.

Date: October 31, 2026.

The histories can get longer and the error can shrink; the transformer and its feedforward networks do not have to get deeper, and the embedding dimension stays put.

Gradient descent is under no obligation to follow our construction. We therefore ask whether trained models arrive at similar computations. We test this in a topic process with observed resets (Section 8), where the construction accumulates ordinary-symbol counts and subtracts those preceding the last reset. After training transformers on next-token prediction, we intervene on their activations and compare the changes in their predictions with those prescribed by the construction. The interventions support an approximate implementation of the accumulation and reset subtraction prescribed by the construction.

Our broader gamble is that models learn reusable algorithms—modular counting, for instance—and put them to work computing beliefs. Finding a counter is a nice start. What is it counting, and what does the model infer from the answer? We hope an algebraic account can connect the algorithms a model learns to the roles they play, and explain how they are reused and combined. For AI safety, the payoff would be to trace a model's reasoning through those computations: what information they track, which beliefs depend on them, and where a correct computation supports an incorrect inference.

1. Prediction and recurrence

A d-dimensional generalized hidden Markov model, GHMM for short, $(\Sigma, \mu, (T_x)_{x\in\Sigma}, \ell)$ consists of a finite *alphabet* Σ, an *initial* row $\mu \in \mathbb{R}^{1\times d}$, a *readout* column $\ell \in \mathbb{R}^{d\times 1}$, and for each $x \in \Sigma$ a *transition matrix* $T_x \in \mathbb{R}^{d\times d}$, such that

$$\mu\ell = 1, \qquad \sum_x T_x\ell = \ell, \qquad \mu T_{x_1}\cdots T_{x_n}\ell \geq 0 \text{ for all } x_1\cdots x_n \in \Sigma.$$

This collection of matrices extend to the free monoid Σ^* (the set of *words*) by

$$T_\varnothing = I, \qquad \text{and } T_w = T_{x_1}\cdots T_{x_n}, \text{ for } w = x_1\cdots x_n \in \Sigma^*.$$

Under these conditions, the numbers $p(w) = \mu T_w\ell$ are consistent cylinder probabilities:

$$p(\varnothing) = 1, \qquad \sum_{x\in\Sigma} p(wx) = p(w) \quad (w \in \Sigma^*).$$

We assume throughout that the GHMM realization $V = \mathbb{R}^{1\times d}$ is minimal (in dimension), so the rows μT_w span V, and the columns $T_w\ell$ span its dual.

For a word w with $p(w) > 0$, define the belief

$$b_w = \frac{\mu T_w}{\mu T_w \ell}. \tag{1.1}$$

In an ordinary hidden Markov model, this is the posterior distribution of the hidden state. In a general model, the coordinates of the belief vector might not be probabilities. They still have exact predictive meaning: for every future word v,

$$\Pr(v \mid w) = \frac{p(wv)}{p(w)} = b_w T_v \ell. \tag{1.2}$$

Thus b_w determines the entire conditional future law, including the optimal next-symbol probabilities. Conversely, two beliefs with the same conditional future law are equal.

If a new later x is observed after w, then the belief is updated by

$$b_{wx} = \frac{b_w T_x}{b_w T_x \ell}, \qquad p(wx) > 0. \tag{1.3}$$

In other words, the current belief and the next observed token determine the next belief. We will use the language of semiautomata to describe this recurrent nature of belief updating, and study when a transformer can simulate this process using a number of layers independent of the length of the word.

2. Algebraic decomposition

We first show that any matrix monoid divides a wreath product of specified factors. We then apply it to a GHMM and recover its belief transformation semigroup as a further divisor of the same wreath product.

2.1. **Transformation semigroups, wreath products and division.**

Definition 2.1 (Transformation semigroup)**.** For a nonempty set X, let $\mathrm{Map}(X)$ be the monoid of all maps $X \to X$. We write maps on the right, with multiplication specified by

$$x(st) = (xs)t \qquad (x \in X,\ s,t \in \mathrm{Map}(X)).$$

A *transformation semigroup* is a pair (X, M) with $M \subseteq \mathrm{Map}(X)$ a subsemigroup. It is a *transformation monoid* if $\mathrm{id}_X \in M$.

Notice that the action is faithful, because elements of M are maps of X. We use transformation monoids below, which is enough because a transformation semigroup can be enlarged to a monoid by adjoining the identity map.

Definition 2.2 (Wreath product)**.** Let (X_1, M_1), (X_2, M_2) and (X_3, M_3) be transformation semigroups. Their *wreath product*

$$(X_1, M_1) \wr (X_2, M_2) \wr (X_3, M_3) = (X_1 \times X_2 \times X_3, W)$$

is defined as follows. The semigroup W consists of triples (f_1, f_2, f_3), where

$$f_1 \in M_1, \qquad f_2 : X_1 \longrightarrow M_2, \qquad f_3 : X_1 \times X_2 \longrightarrow M_3.$$

The action of W on $X_1 \times X_2 \times X_3$ is

$$(x_1, x_2, x_3)(f_1, f_2, f_3) = \big(x_1 f_1,\ x_2 f_2(x_1),\ x_3 f_3(x_1, x_2)\big).$$

Multiplication in W is defined by

$$(f_1, f_2, f_3)(f_1', f_2', f_3') = (f_1 f_1', F_2, F_3),$$

where $F_2 : X_1 \to M_2$ and $F_3 : X_1 \times X_2 \to M_3$ are given by

$$F_2(x_1) = f_2(x_1)\, f_2'(x_1 f_1), \qquad F_3(x_1, x_2) = f_3(x_1, x_2)\, f_3'\big(x_1 f_1, x_2 f_2(x_1)\big).$$

If the factors M_i are monoinds, then so is W with identity (f_1, f_2, f_3) where $f_1 = 1_{M_1}$ and f_2, f_3 constant with values $1_{M_2}, 1_{M_3}$, respectively.

This definition can be generalized to any finite number of factors, and the wreath transformation applied to each coordinate depends on the original values of all preceding coordinates. We call an action generated by specified elements of a wreath product a *cascade*.

Definition 2.3 (Division)**.** A transformation semigroup (X, M) *divides* (Y, L), written $(X, M) \mid (Y, L)$, if there are a subsemigroup $L_0 \leq L$, a nonempty subset $Y_0 \subseteq Y$ which is L_0-invariant (so $Y_0 L_0 \subseteq Y_0$), and surjections

$$q_1 : Y_0 \twoheadrightarrow X, \qquad q_2 : L_0 \twoheadrightarrow M,$$

such that q_2 is a semigroup homomorphism and

$$q_1(y\ell) = q_1(y) q_2(\ell) \qquad (y \in Y_0,\ \ell \in L_0).$$

In this case, we call (Y, L) a *cover* of (X, M). For transformation monoids we require L_0 to contain the identity and q_2 to preserve it.

Intuitively, $(X, M) \mid (Y, L)$ means that we can recover (X, M) in terms of (Y, L) by first restricting the underlying set and transformations, then taking compatible quotients. Division is transitive.

2.2. **The factors of the decomposition.** Here are some of the factors needed for the decomposition

Definition 2.4 (Regular actions and resets)**.** For $a \in X$, write $K_a : X \to X$ for the constant map $x \mapsto a$. We use the following constructions:

(i) The *reset action* on X is $(X, \mathcal{M}(X))$, where $\mathcal{M}(X) = \{\mathrm{id}_X\} \cup \{K_a : a \in X\}$.
(ii) The *regular action* of a monoid S is (S, S), where $h \in S$ acts by

$$R_h : S \longrightarrow S, \qquad R_h(s) = sh.$$

(iii) For a group H, its *regular action with resets* is $(H, \mathcal{G}(H))$, where

$$\mathcal{G}(H) = \{K_h : h \in H\} \cup \{R_h : h \in H\}.$$

Next we introduce the groups used in the decomposition, which come from the standard action of matrices on (row) vectors. Fix a submonoid $\mathcal{S} \subseteq \mathrm{Mat}_d(\mathbb{R})$, acting on $V = \mathbb{R}^{1\times d}$.

Definition 2.5 (Eventual image and stabilizer group)**.** A subspace $P \subseteq V$ is an *eventual image* of $\mathcal{S}$ if

$$P = VA, \qquad PA = P \qquad \text{for some } A \in \mathcal{S}.$$

Equivalently, $P = VA$ is the image of some A with $\mathrm{rank}(A^2) = \mathrm{rank}\, A$.

The *stabilizer group* of such P is

$$H_P = \langle C|_P : C \in \mathcal{S},\ PC = P \rangle \subseteq \mathrm{GL}(P). \tag{2.6}$$

Such restrictions $C|_P$ are invertible on P. The brackets mean the subgroup they generate.

2.3. **The matrix decomposition.** The next result is of independent mathematical interest. Its construction and proof are given in Section 4.

Theorem 2.7 (Cascade decomposition)**.** *For any submonoid $\mathcal{S} \leq \mathrm{Mat}_d(\mathbb{R})$, the regular action $(\mathcal{S}, \mathcal{S})$ divides a wreath product of finitely many factors, each of the following form:*

(i) *a reset action $(X, \mathcal{M}(X))$;*
(ii) *the regular action with resets $(H_P, \mathcal{G}(H_P))$ for an eventual image P of $\mathcal{S}$;*
(iii) *a finite transformation monoid whose subgroups have order at most two.*

2.4. **The belief semiautomaton.**

Definition 2.8 (Semiautomata and their monoids)**.** A *semiautomaton* is a triple $(Q, \Sigma, (f_x)_{x\in\Sigma})$, where Q is a nonempty set of *states*, Σ is an *alphabet* (with elements called *letters*), and $f_x : Q \to Q$ is a map for each $x \in \Sigma$, called *transitions*.

Its associated transformation monoid is (Q, M) with

$$M = \{f_w : w \in \Sigma^*\} \subseteq \mathrm{Map}(Q),$$

where for a word $w = x_1 \cdots x_n \in \Sigma^*$, we denote $f_w = f_{x_1} \cdots f_{x_n}$, and $f_1 = \mathrm{id}_X$ for the empty word $1 = 1_{\Sigma^*}$.

Now take a GHMM $(\Sigma, \mu, (T_x)_{x\in\Sigma}, \ell)$ as in Section 1.

Definition 2.9 (Belief semiautomaton of a GHMM)**.** Consider the set of beliefs $B = \{b_w : p(w) > 0\}$. If some word has probability zero, adjoin a sink $\perp$ and put $B_\perp = B \sqcup \{\perp\}$; otherwise put $B_\perp = B$. For each $x \in \Sigma$, define $f_x : B_\perp \to B_\perp$ by

$$f_x(b) = \begin{cases} bT_x/(bT_x\ell), & bT_x\ell > 0, \\ \perp, & bT_x\ell = 0, \end{cases} \qquad f_x(\perp) = \perp .$$

The *belief semiautomaton* of $(\Sigma, \mu, (T_x)_{x\in\Sigma}, \ell)$ is $(B_\perp, \Sigma, (f_x)_{x\in\Sigma})$.

Hence, for a GHMM $(\Sigma, \mu, (T_x)_{x\in\Sigma}, \ell)$, we have two transformation monoids:

(1) the *linear* monoid of matrices $\mathcal{S} = \{T_w : w \in \Sigma^*\}$ with its regular action; and
(2) the transformation monoid $(B_\perp, M_B)$ associated to its belief semiautomaton.

The linear one, $\mathcal{S}$, is richer in structure, which allows us to proof Theorem 2.7. However, the relevant one for us is the belief semiautomaton $(B_\perp, M_B)$, because it carries predictive meaning and describes Bayesian updates. The next result explains how they are related under division (Definition 2.3).

Theorem 2.10 (Decomposition of the belief action)**.** *Consider a GHMM $(\Sigma, \mu, (T_x)_{x\in\Sigma}, \ell)$.*

The transformation monoid $(B_\perp, M_B)$ of the belief semiautomaton is a quotient of the regular action $(\mathcal{S}, \mathcal{S})$. The quotient sends T_w to b_w when $p(w) > 0$ and to the sink otherwise.

Proof. Consider the maps $q_1 \colon \mathcal{S} \to B_\perp$ and $q_2 \colon \mathcal{S} \to M_B$ given by

$$q_1(A) = \begin{cases} \mu A/(\mu A\ell), & \mu A\ell > 0, \\ \perp, & \mu A\ell = 0, \end{cases} \qquad q_2(T_w) = f_w.$$

The transformation f_w depends only on T_w: it sends a belief b to $bT_w/(bT_w\ell)$ when the denominator is positive, and to the sink otherwise. Thus q_2 is well-defined, and it clearly is a monoid homomorphism. Cancellation of the normalization gives

$$q_1(AC) = q_1(A)q_2(C) \qquad (A, C \in \mathcal{S}). \tag{2.11}$$

This identity also holds when a denominator vanishes, because an impossible word has only impossible extensions, by consistency and nonnegativity of the word probabilities. Hence, these maps give the quotient, and transitivity of division completes the proof. □

As a consequence of Theorem 2.10, the beliefs object $(B_\perp, M_B)$ also divides the wreath product supplied by Theorem 2.7. Thus, these two results provide an algebraic description of the Bayesian updates process.

3. Constant depth under solvability

3.1. **Our models.** An L-layer transformer is a map $\mathcal{T} : (\mathbb{R}^m)^{n+1} \to (\mathbb{R}^m)^{n+1}$ of the form

$$\mathcal{T} = B_L \circ \cdots \circ B_1 \circ (\mathrm{id} + P), \qquad B_j = (\mathrm{id} + F_j \circ \mathcal{N}_j^{(F)}) \circ (\mathrm{id} + A_j \circ \mathcal{N}_j^{(A)}).$$

Here P is the *positional encoding* and each B_j is one layer with the following components:

- A_j is *causal multihead softmax attention.*
- F_j is a *ReLU network* (the same network is applied to each position).
- $\mathcal{N}_j^{(A)}, \mathcal{N}_j^{(F)}$ apply *LayerNorm* independently to each token's residual vector.

This defines the *core* of $\mathcal{T}$. The full network, including its input embedding and affine readout, and our definition of simulation are given in Section 5.

Our softmax and ReLU blocks follow [2, Appendix A.4]; we additionally use LayerNorm. Our analysis includes intermediate approximation errors but assumes that the network operations are evaluated without floating-point roundoff.

Theorem 3.1 (Constant-depth belief simulation)**.** *Let $\mathcal{P} = (\Sigma, \mu, (T_x)_{x\in\Sigma}, \ell)$ be a fixed GHMM of dimension d whose eventual-image stabilizers are solvable. For every $N \geq 1$ and $0 < \varepsilon \leq 1$, its belief semiautomaton admits an ε-simulation at horizon N by a transformer as in Section 5, with depth L, feedforward depth K and embedding dimension m satisfying*

$$L \leq 10d + \sum_{r:q_r>0} (6 + \lambda_r + \tau_r), \qquad K \leq 4, \qquad m = O_{\mathcal{P}}(1),$$

and using $\widetilde{O}_{\mathcal{P}}((N + \log(1/\varepsilon) + 2)^{c_{\mathcal{P}}})$ heads. The remaining complexity measures are polynomial in $N + \log(1/\varepsilon) + 2$. Support detection and the sink output are exact.

Here q_r counts the recurrent classes at rank r, λ_r is the largest derived length of their finite quotient groups $H_{P_\kappa}/H^0_{P_\kappa}$, and τ_r is the largest triangular height $\tau(\Omega_{r,\kappa})$, using supports over all original words. The exponent $c_{\mathcal{P}}$ is given in (D.3); $\widetilde{O}$ suppresses logarithmic factors. Section 7 explains the construction; Appendix D gives the implementation and resource bounds.

4. Wreath-product decomposition of linear monoids

This section is dedicated to the proof of Theorem 2.7. This is done in two stages. We first obtain a decomposition by rank, and then each rank factor is further decomposed.

4.1. **Rank decomposition.** Fix $d \geq 1$ and a submonoid $\mathcal{S} \leq \mathrm{Mat}_d(\mathbb{R})$. Here we show that $\mathcal{S}$ (with its regular action) divides a wreath product of $(X_d, M_d), \ldots, (X_0, M_0)$ where the transformation semigroup (X_r, M_r) deals with the elements of $\mathcal{S}$ of rank r.

Definition 4.1 (Rank actions)**.** For $0 \leq r \leq d$, set

$$J_r = \{\varnothing\} \sqcup \{A \in \mathcal{S} : \operatorname{rank} A \leq r\}, \qquad X_r = \{\varnothing\} \sqcup \{A \in \mathcal{S} : \operatorname{rank} A = r\}.$$

Here $\varnothing$ is a formal empty product, with $\varnothing A = A\varnothing = A$ and $\varnothing\varnothing = \varnothing$. For $E \in J_r$, define

$$\theta_E^{(r)} : X_r \longrightarrow X_r, \qquad y\theta_E^{(r)} = \begin{cases} yE, & yE \in X_r, \\ \varnothing, & yE \notin X_r. \end{cases}$$

The *rank-r action* is (X_r, M_r), where

$$M_r = \langle \theta_E^{(r)} : E \in J_r \rangle \leq \mathrm{Map}(X_r).$$

Proposition 4.2 (Rank decomposition)**.** *The regular action of $\mathcal{S}$ divides the wreath product of its rank actions:*

$$(\mathcal{S}, \mathcal{S}) \mid (X_d, M_d) \wr \cdots \wr (X_0, M_0).$$

The division recovers matrices by restricting the map

$$\pi : X_d \times \cdots \times X_0 \longrightarrow \mathcal{S}, \qquad \pi(y_d, \ldots, y_0) = y_0 y_1 \cdots y_d$$

to an orbit of $(\varnothing, \ldots, \varnothing)$, where the product on the right is evaluated in $\mathcal{S}$ by replacing each $\varnothing$ with the identity matrix.

Proof. Write $(Y, L) = (X_d, M_d) \wr \cdots \wr (X_0, M_0)$ and $y_* = (\varnothing, \ldots, \varnothing) \in Y$.

Following Definition 2.3, we must construct a submonoid $L_0 \leq L$, a nonempty L_0-invariant subset $Y_0 \subseteq Y$, and surjections $q_1 : Y_0 \to \mathcal{S}$ and $q_2 : L_0 \to \mathcal{S}$ such that q_2 is a monoid homomorphism and $q_1(yu) = q_1(y)q_2(u)$.

Step 1. We construct L_0 by specifying a transformation $\widetilde{A} \in L$ for each $A \in \mathcal{S}$. Given $y = (y_d, \ldots, y_0) \in Y$, we define an element E_r that will specify the map $\theta_{E_r}^{(r)} \in M_r$ to be applied to y_r. Define $E_d, \ldots, E_0$ recursively by $E_d = A$ and

$$E_{r-1} = \begin{cases} \varnothing, & y_r E_r \in X_r, \\ y_r E_r, & y_r E_r \notin X_r, \end{cases} \qquad r = d, d-1, \ldots, 1. \tag{4.3}$$

We check that $E_r \in J_r$. Inductively, this holds for $E_d = A$. If $y_r E_r \notin X_r$, then $E_r \neq \varnothing$, and $\operatorname{rank}(y_r E_r) \leq r$ but $\operatorname{rank}(y_r E_r) \neq r$; hence $y_r E_r \in J_{r-1}$. In the case $y_r E_r \in X_r$ we have $E_{r-1} = \varnothing \in J_{r-1}$.

Further, E_r depends only on A and the preceding entries $y_d, \ldots, y_{r+1}$. Hence, the rule

$$y\widetilde{A} = (y'_d, \ldots, y'_0), \qquad y'_r = y_r \theta_{E_r}^{(r)}$$

defines an element $\widetilde{A} \in L$: the transformation chosen in each coordinate is only a function of the preceding coordinates, as required by Definition 2.2. Set

$$L_0 = \langle \widetilde{A} \colon A \in \mathcal{S} \rangle \leq L.$$

Step 2. We define the map q_1. Take $Y_0 = y_* L_0$. This set contains y_* and is L_0-invariant, since $(y_* u)v = y_*(uv)$ for $u, v \in L_0$. Define $q_1 = \pi|_{Y_0} : Y_0 \to \mathcal{S}$. To prove surjectivity, we show that $q_1(y_* \widetilde{A}) = A$ for every $A \in \mathcal{S}$. We first compute $\pi(y\widetilde{A})$. For $1 \leq r \leq d$, the two branches of (4.3) give

$$E_{r-1} y_r' = y_r E_r.$$

In the first branch $E_{r-1} = \varnothing$ and $y_r' = y_r E_r$; in the second, $E_{r-1} = y_r E_r$ and $y_r' = \varnothing$. At rank $r = 0$ zero, multiplication stays in X_0, so $y_0' = y_0 E_0$. It follows that

$$\pi(y\widetilde{A}) = y_0' y_1' \cdots y_d' = y_0 E_0 y_1' \cdots y_d' = y_0 y_1 E_1 y_2' \cdots y_d' = \cdots = y_0 y_1 \cdots y_d A.$$

We have therefore proved

$$\pi(y\widetilde{A}) = \pi(y) A \qquad (y \in Y,\ A \in \mathcal{S}). \tag{4.4}$$

Since $\pi(y_*) = I$, this gives $q_1(y_* \widetilde{A}) = A$ for every $A \in \mathcal{S}$, so q_1 is surjective.

Step 3. We define the map q_2. Let

$$q_2 : L_0 \longrightarrow \mathcal{S}, \quad q_2(\widetilde{A}_1 \cdots \widetilde{A}_k) = A_1 \cdots A_k,$$

with the empty product sent to I. To check that q_2 is well-defined, suppose two products of the transformations $\widetilde{A}$ give the same element of L_0. They then agree at y_*, and applying π and (4.4) shows that their matrix products are equal. Concatenation shows that q_2 is a monoid homomorphism, and $q_2(\widetilde{A}) = A$ shows that it is surjective.

Finally, the maps are compatible: iterating (4.4) gives

$$q_1(yu) = q_1(y) q_2(u) \qquad (y \in Y_0,\ u \in L_0).$$

This proves the required division. □

4.2. **Decomposition of the rank actions.** Recall that $\mathcal{S} \leq \mathrm{Mat}_d(\mathbb{R})$ and we denote $V = \mathbb{R}^{1 \times d}$. We now refine the decomposition of Proposition 4.2, by furthere decomposiong (X_r, M_r) for each rank r. Throughout the argument, fix $1 \leq r \leq d$ and write $D_r = \binom{d}{r}$. The rank-zero action is treated at the end.

To organize this decomposition, we study certain *rank preserving* sequences $(A_1, \ldots, A_n)$ of matrices of rank r, then we study how to partition them into *blocks*, and finally how to multiply the elements in each block using the stabilizer groups H_P.

4.2.1. *Rank preserving sequences and block decomposition.*

Definition 4.5 (Classes of recurrent matrices)**.** For matrices $A, B \in \mathcal{S}$ of rank r, write

$$A \rightsquigarrow B \quad \Longleftrightarrow \quad \mathrm{rank}(ACB) = r \text{ for some } C \in \mathcal{S}.$$

A rank-r matrix A is *recurrent* if $A \rightsquigarrow A$. Two recurrent matrices are *equivalent* if $A \rightsquigarrow B$ and $B \rightsquigarrow A$. Their equivalence classes will be called *recurrent classes*.

The relation $\rightsquigarrow$ is transitive: if ACB and BED have rank r, then $VACB = VB$. Thus

$$V(ACBED) = (VB)ED = VBED$$

has dimension r. The matrix $CBE \in \mathcal{S}$ therefore witnesses $A \rightsquigarrow D$. Thus mutual reachability is an equivalence relation on recurrent matrices.

Remark 4.6. We will use exterior powers to turn rank-r matrices into rank-one matrices. For $C \in \mathrm{Mat}_d(\mathbb{R})$, let $C^{[r]}$ denote its action on $\bigwedge^r V$. In the standard basis its entries are the $r \times r$ minors of C, and

$$(CD)^{[r]} = C^{[r]}D^{[r]}, \qquad \dim \textstyle\bigwedge^r V = D_r, \qquad \operatorname{rank} C^{[r]} = \binom{\operatorname{rank} C}{r}.$$

The last identity follows by choosing bases of the image and kernel of C. Thus $C^{[r]} = 0$ if $\operatorname{rank} C < r$. When $\operatorname{rank} C = r$, the matrix $C^{[r]}$ has rank one, so it admits a factorization

$$C^{[r]} = uv, \qquad u \in \mathbb{R}^{D_r \times 1}, \quad v \in \mathbb{R}^{1 \times D_r}, \quad u, v \neq 0.$$

Lemma 4.7 (Finite recurrent data)**.** (i) *Each recurrent class of rank r contains a matrix E with $\operatorname{rank}(E^2) = r$. Hence $P = VE = PE$ is an eventual image.*

(ii) *There are at most D_r recurrent classes of rank r.*

(iii) *There is a set $W_r \subseteq \mathcal{S}$, with $|W_r| \leq D_r^2$, such that, for all rank-r matrices $A, B \in \mathcal{S}$,*

$$A \rightsquigarrow B \quad \Longleftrightarrow \quad \operatorname{rank}(ACB) = r \text{ for some } C \in W_r$$

If $\mathcal{S}$ is finitely generated, the elements of W_r can be chosen as products of at most D_r^2 generators.

Proof. (i) If $\operatorname{rank}(ACA) = r$, put $E = CA$. Then $VCA = VA$ and $VACA = VA$, whence $VE^2 = VA = VE$. Moreover, $A \rightsquigarrow E$ is witnessed by I, and $E \rightsquigarrow A$ by C, because $AE = ACA$ and $ECA = E^2$ have rank r. Thus E belongs to the class of A.

(ii) Take any finite collection of distinct classes, say m of them. Take recurrent representatives $E_1, \ldots, E_m$, with $\operatorname{rank}(E_i^2) = r$. Transitivity allows them to be ordered so that $E_i \not\rightsquigarrow E_j$ whenever $i > j$ (a directed cycle would put its members in one class). By Remark 4.6, write

$$E_i^{[r]} = u_i v_i, \qquad u_i \in \mathbb{R}^{D_r \times 1}, \quad v_i \in \mathbb{R}^{1 \times D_r}.$$

Here $v_i u_i \neq 0$, because E_i^2 has rank r, whereas $v_i u_j = 0$ for $i > j$, because otherwise $\operatorname{rank}(E_i E_j) = r$. Therefore, the matrix $(v_i u_j)_{i,j=1}^m$ is triangular with nonzero diagonal entries, so its rank is m. On the other hand, the factorization

$$(v_i u_j)_{i,j=1}^m = \underbrace{\begin{pmatrix} v_1 \\ \vdots \\ v_m \end{pmatrix}}_{m \times D_r} \underbrace{\begin{pmatrix} u_1 & \cdots & u_m \end{pmatrix}}_{D_r \times m}$$

shows that its rank is at most D_r. Thus every finite collection of classes has size $m \leq D_r$, which proves the bound.

(iii) The space $\operatorname{span}\{C^{[r]} : C \in \mathcal{S}\}$ is a subspace of $\mathrm{Mat}_{D_r}(\mathbb{R})$, so its dimension is at most D_r^2. Choose W_r so that $\{C^{[r]} : C \in W_r\}$ is a basis of $\operatorname{span}\{C^{[r]} : C \in \mathcal{S}\}$. For fixed $A, B \in \mathcal{S}$ of rank r, the map

$$\mathrm{Mat}_{D_r}(\mathbb{R}) \longrightarrow \mathrm{Mat}_{D_r}(\mathbb{R}), \qquad Z \longmapsto A^{[r]} Z B^{[r]}$$

is linear. It vanishes on this entire span exactly when it vanishes on the chosen basis. Since $\operatorname{rank}(ACB) \le r$, this proves that $A \rightsquigarrow B$ precisely when $\operatorname{rank}(ACB) = r$ for some $C \in W_r$.

Finally, suppose $\mathcal{S}$ is finitely generated. We show that the elements of W_r can be chosen as products of at most D_r^2 generators. Let U_n be the span of exterior matrices of words of length at most n (in the chosen generators). If $U_n = U_{n+1}$, multiplication by each generator preserves U_n, so all subsequent spans are equal. Since $\dim U_0 = 1$ and $\dim U_n \le D_r^2$, a basis is obtained by length at most D_r^2. □

Notice that the proof of (ii) is inspired by [6, Proposition 8].

Left multiplication by an invertible $G \in \mathcal{S}$ preserves recurrence and recurrent classes. Indeed, if GM is recurrent, a witness C for $\operatorname{rank}(GMCGM) = \operatorname{rank} M$ gives the witness CG for recurrence of M. Conversely, suppose $\operatorname{rank}(MCM) = r = \operatorname{rank} M \ge 1$. Cayley–Hamilton expresses the identity as a linear combination of $G^{[r]}, \dots, (G^{[r]})^{D_r}$. Hence $\operatorname{rank}(MCG^nM) = r$ for some $1 \le n \le D_r$. The witnesses CG^{n-1} and C give $M \rightsquigarrow GM$ and $GM \rightsquigarrow M$, respectively. Transitivity proves the assertion.

Definition 4.8 (Block decomposition)**.** Let $(A_1, \dots, A_n)$ be a sequence in $\mathcal{S}$.

(i) The sequence is *rank-preserving at rank r* if $\operatorname{rank} A_i = \operatorname{rank}(A_1 \cdots A_n) = r$ for each i.
(ii) A *block* is either a maximal consecutive subsequence $(A_i, \dots, A_j)$ whose matrices all belong to the same recurrent class, or a singleton (A_i) with A_i nonrecurrent.
(iii) These blocks partition the sequence, since each index belongs to exactly one block. This partition is its *block decomposition*.

Lemma 4.9 (Number of blocks)**.** *The block decomposition of a rank-preserving sequence at rank r has at most $K_r = 2D_r - 1$ blocks. No recurrent class occurs in two different blocks.*

Proof. Let $(A_1, \dots, A_n)$ be a rank-preserving sequence at rank r. Note that every consecutive subproduct also has rank r, because multiplication cannot increase rank. In particular $A_i \rightsquigarrow A_j$ whenever $i < j$. We count the two kinds of blocks (recurrent and not recurrent) separately.

Step 1: Counting blocks belonging to recurrent classes. In fact, if A_i and A_j belong to the same recurrent class and $i < k < j$, then

$$A_i \rightsquigarrow A_k \rightsquigarrow A_j \rightsquigarrow A_i.$$

Thus A_k is recurrent and belongs to that class as well. By maximality, A_i and A_j lie in the same block. Hence each recurrent class contributes at most one block, giving at most D_r such blocks by Lemma 4.7. Notice that this includes singletons of recurrent matrices.

Step 2: Counting singleton blocks of nonrecurrent matrices. Let q be their number, and let $i_1 < \cdots < i_q$ be their positions. If $q = 0$, Step 1 proves the result. Assume $q \ge 1$.

Substep 2.1: Constructing nonrecurrent products of rank r. Define

$$B_j = A_{i_j} \cdots A_{i_{j+1}-1} \quad (1 \le j < q), \qquad B_q = A_{i_q} \cdots A_n.$$

Each product starts at a nonrecurrent matrix and ends immediately before the next one, or at A_n if there is no next one. Each B_j has rank r and is nonrecurrent. To see the latter, write $B_j = A_{i_j} D$. If $\operatorname{rank}(B_j C B_j) = r$ for some $C \in \mathcal{S}$, then $B_j C B_j = (A_{i_j} D C A_{i_j}) D$ forces $\operatorname{rank}(A_{i_j} D C A_{i_j}) = r$, because $\operatorname{rank}(A_{i_j}) = r$ and multiplication cannot increase rank. This contradicts nonrecurrence of A_{i_j}.

Substep 2.2: Factoring the exterior powers and establishing their pairings. By Remark 4.6, write $B_j^{[r]} = u_j v_j$ with nonzero $u_j \in \mathbb{R}^{D_r \times 1}$ and $v_j \in \mathbb{R}^{1 \times D_r}$. We claim that

$$v_j u_{j+1} \neq 0 \quad (j < q), \qquad v_l u_k = 0 \quad (l \ge k).$$

The first assertion follows from $\operatorname{rank}(B_j B_{j+1}) = r$. For the second, a nonzero pairing would give $\operatorname{rank}(B_l B_k) = r$. If $l > k$, since $V B_k \cdots B_l = V B_l$, the product $B_k \cdots B_l B_k$ would then have rank r, making B_k recurrent. For $l = k$, use B_k^2 to get the same contradiction.

Substep 2.3: Bounding q. The columns $u_1, \ldots, u_q$ are linearly independent: in a linear relation, apply $v_{q-1}, v_{q-2}, \ldots, v_1$ successively to eliminate the coefficients of $u_q, u_{q-1}, \ldots, u_2$, then use $u_1 \neq 0$. All these columns lie in $\ker v_q$, of dimension $D_r - 1$. Thus $q \le D_r - 1$.

Together with Step 1, this gives at most $D_r + (D_r - 1) = K_r$ blocks in total. □

4.2.2. *Multiplication within a block.* Consider a rank-preserving block $(A_1, \ldots, A_n)$ whose matrices belong to a recurrent class κ. Each multiplication map gives an isomorphism

$$V A_i \longrightarrow V A_{i+1}, \qquad v \longmapsto v A_{i+1} \qquad (1 \le i < n),$$

even though the image spaces may differ. We will identify them with one fixed eventual image, so that these isomorphisms can be expressed as elements of its stabilizer group.

By Lemma 4.7, choose $E_\kappa \in \kappa$ with $\operatorname{rank}(E_\kappa^2) = r$, and put $P_\kappa = V E_\kappa$. For each $A \in \kappa$, choose $C_A \in W_r$ such that $\operatorname{rank}(A C_A E_\kappa) = r$; such a choice exists by Lemma 4.7, since $A \rightsquigarrow E_\kappa$. Right multiplication by $C_A E_\kappa$ defines an isomorphism

$$\beta_A : V A \longrightarrow P_\kappa, \qquad v \longmapsto v C_A E_\kappa. \tag{4.10}$$

Recall that we compose linear maps from left to right, consistently with our right actions.

Proposition 4.11 (Multiplication within a class)**.** *Fix a recurrent class κ at rank r and $E_\kappa \in \kappa$ with $\operatorname{rank}(E_\kappa^2) = r$.*

(i) *If $A, B \in \kappa$ and $\operatorname{rank}(AB) = r$, the maps β_A and β_B of (4.10) satisfy*

$$h(A, B) := \beta_A^{-1}(B|_{VA})\beta_B \in H_{P_\kappa}.$$

(ii) *For a rank-preserving sequence $(A_1, \ldots, A_n)$ with all $A_i \in \kappa$, we have*

$$A_1 \cdots A_n = A_1 \beta_{A_1} \left(\prod_{i=2}^{n} h(A_{i-1}, A_i) \right) \beta_{A_n}^{-1} \tag{4.12}$$

as maps $V \to V$.

(iii) *For another representative $E'_\kappa \in \kappa$ with $\operatorname{rank}((E'_\kappa)^2) = r$, put $Q = V E'_\kappa$. There is an isomorphism $\alpha : P_\kappa \to Q$ such that $\alpha H_Q \alpha^{-1} = H_{P_\kappa}$.*

Proof. Put $P = P_\kappa$, the (eventual) image of E_κ.

(i) Since $E_\kappa \rightsquigarrow A$, we can choose $C \in \mathcal{S}$ with $\operatorname{rank}(E_\kappa CA) = r$. Then the restriction

$$\alpha := (CA)|_P \colon P \longrightarrow VA$$

is an isomorphism. Since $\operatorname{rank}(AB) = r$, multiplication by B is also an isomorphism $VA \to VB$. Consequently, the maps

$$\alpha\beta_A \colon P \to P, \qquad \alpha(B|_{VA})\beta_B \colon P \to P$$

are restrictions of matrices in $\mathcal{S}$ that map P onto itself. They therefore belong to H_P, and hence

$$(\alpha\beta_A)^{-1}(\alpha(B|_{VA})\beta_B) = \beta_A^{-1}(B|_{VA})\beta_B = h(A,B) \in H_P.$$

(ii) For $n = 1$, the empty group product is id_P, and the formula reduces to $A_1\beta_{A_1}\beta_{A_1}^{-1} = A_1$. So assume $n \geq 2$. Notice that every consecutive subproduct has rank r, since the whole sequence is rank-preserving. Now, by definition of h, we have

$$A_{i+1}|_{VA_i} = \beta_{A_i} h(A_i, A_{i+1})\beta_{A_{i+1}}^{-1} \qquad (1 \leq i < n).$$

Substitute these identities into $A_1 \cdots A_n$, using the fact $V(A_1 \cdots A_i) = VA_i$ to restrict each multiplication to VA_i. The intermediate pairs $\beta_{A_i}^{-1}\beta_{A_i} = \mathrm{id}_P$ cancel, giving (4.12).

(iii) Let $E'_\kappa \in \kappa$ satisfy $\operatorname{rank}((E'_\kappa)^2) = r$, and put $Q = VE'_\kappa$. Since $E_\kappa \rightsquigarrow E'_\kappa$ and $E'_\kappa \rightsquigarrow E_\kappa$, choose $C, D \in \mathcal{S}$ such that

$$\alpha = (CE'_\kappa)|_P : P \longrightarrow Q, \qquad \beta = (DE_\kappa)|_Q : Q \longrightarrow P$$

are isomorphisms. For any defining generator $h = T|_Q$ of H_Q (where $T \in \mathcal{S}$ and $QT = Q$), both $\alpha h \beta$ and $\alpha\beta$ are restrictions of matrices in $\mathcal{S}$ mapping P onto itself. Thus

$$\alpha h \alpha^{-1} = (\alpha h \beta)(\alpha\beta)^{-1} \in H_P.$$

Similarly, for a generator $g = U|_P$ of H_P, both $\beta\alpha$ and $\beta g \alpha$ belong to H_Q, so

$$\alpha^{-1} g \alpha = (\beta\alpha)^{-1}(\beta g \alpha) \in H_Q.$$

The first inclusion on generators gives $\alpha H_Q \alpha^{-1} \subseteq H_P$; the second gives the reverse inclusion. Hence $\alpha H_Q \alpha^{-1} = H_P$. □

The factorization in (ii) follows the same principle as the proof of [6, Proposition 13], expressed here using isomorphisms between image spaces and their stabilizer groups.

4.2.3. *Decomposition of one rank.* We introduce one last transformation monoid.

Definition 4.13 (Block bookkeeping)**.** Fix $1 \leq r \leq d$ and put $K = K_r$.

(i) Let Λ be the set of recurrent classes at rank r, together with a symbol $\circ$. For a rank-r matrix $A \in \mathcal{S}$, define $\kappa(A) \in \Lambda$ to be its recurrent class if A is recurrent, and $\circ$ otherwise. Set

$$\mathscr{C} = \{0\} \sqcup (\{1, \ldots, K\} \times \Lambda).$$

(ii) For $A, E \in J_r$, define $\gamma_{A,E} : \mathscr{C} \to \mathscr{C}$ as follows. If $E = \varnothing$, take $\gamma_{A,E} = \mathrm{id}_{\mathscr{C}}$. If $E \neq \varnothing$ and $\operatorname{rank} E < r$, take $\gamma_{A,E}$ as the constant map with value 0. If $\operatorname{rank} E = r$, put $0\gamma_{A,E} = (1, \kappa(E))$, and, for $1 \leq k \leq K$ and $\kappa \in \Lambda$, put

$$(k,\kappa)\gamma_{A,E} = \begin{cases} 0, & \operatorname{rank}(AE) < r, \\ (k,\kappa), & \operatorname{rank}(AE) = r,\ \kappa \neq \circ,\ A, E \in \kappa, \\ (\min\{k+1, K\}, \kappa(E)), & \text{otherwise.} \end{cases}$$

(iii) Define the transformation monoid $(\mathscr{C}, U_r)$ by

$$U_r = \langle \gamma_{A,E} : A, E \in J_r \rangle \leq \operatorname{Map}(\mathscr{C}).$$

Lemma 4.14. *The monoid U_r is finite, and every subgroup of U_r has order at most two.*

Proof. The set Λ is finite by Lemma 4.7, hence so are $\mathscr{C}$ and U_r. It remains to bound the orders of its subgroups.

Step 1: Maps with more than two outputs. Define

$$\nu \colon \mathscr{C} \longrightarrow \{0, \ldots, K\}, \qquad \nu(0) = 0, \quad \nu(k,\kappa) = k.$$

For $A, E \in J_r$ with $E \neq \varnothing$, the map $\gamma_{A,E}$ has image $\{0\}$ if $\operatorname{rank} E < r$, and image $\{0, (1, \kappa(E))\}$ if $\operatorname{rank} E = r > \operatorname{rank}(AE)$. In the remaining case, its image is contained in $\{1, \ldots, K\} \times \{\kappa(E)\}$ and $\nu(t\gamma_{A,E}) \geq \nu(t)$ for every $t \in \mathscr{C}$.

Every $u \in U_r$ is a composition of maps $\gamma_{A,E}$. A composition containing a map with at most two outputs also has at most two outputs. Thus, if $u \neq \mathrm{id}_{\mathscr{C}}$ and $|\mathscr{C}u| > 2$, every factor, after identities are omitted, must satisfy the remaining case above. The inequalities persist under composition, and the last factor determines the label of every output. Therefore

$$\mathscr{C}u \subseteq \{1, \ldots, K\} \times \{\kappa\} \quad \text{for some } \kappa \in \Lambda, \qquad \nu(tu) \geq \nu(t) \quad \text{for all } t \in \mathscr{C}.$$

Step 2: Representing a subgroup by permutations. Let $G \leq U_r$ have identity e, and put $R = \mathscr{C}e$. By the restriction argument in [7, Section 3], G acts faithfully on R by permutations. Since $eg = g$, we also have $\mathscr{C}g = Rg = R$ for every $g \in G$.

Step 3: Bounding the order of such G. If $|R| \leq 2$, Step 2 embeds G in the symmetric group on at most two points, so $|G| \leq 2$. Suppose $|R| > 2$. We show that G is trivial. Otherwise, choose $u \in G$ with $u \neq \mathrm{id}_{\mathscr{C}}$. By Step 1, $R = \mathscr{C}u$ lies in $\{1, \ldots, K\} \times \{\kappa\}$ for one κ, so ν is injective on R. Every $g \in G$ permutes R by Step 2 and satisfies $\nu(tg) \geq \nu(t)$ by Step 1 (also when $g = \mathrm{id}_{\mathscr{C}}$). Along each cycle of this permutation, these inequalities force all values of ν to be equal. Injectivity of ν then forces g to fix every point of R. By faithfulness, G is trivial, a contradiction. □

In the construction below, $0 \in \mathscr{C}$ means that no block is currently stored. A pair (k, κ) records the number of blocks since the last reset, including the current block, and its label κ. With A the last matrix read and E the next input, $\gamma_{A,E}$ updates these data: continuing the current block leaves them unchanged, starting a new block increases the count and assigns the label $\kappa(E)$, and a rank drop resets them to 0. Empty inputs change nothing.

We now combine the preceding results to decompose (X_r, M_r). By Lemma 4.9, there are at most K_r blocks in a rank-preserving sequence at rank r, so a direct product of $K_r - 1$ reset actions suffices to store the products of the completed blocks. If the current block belongs to a recurrent class, Proposition 4.11 represents its product using its first and last matrices and one element of the corresponding stabilizer group. A nonrecurrent block consists of a single matrix, which we store directly. The finite action of Definition 4.13 determines which group and reset factors to update.

Fix $1 \le r \le d$. For each recurrent class κ at rank r, choose $E_\kappa \in \kappa$ with $\operatorname{rank}(E_\kappa^2) = r$, and put $P_\kappa = VE_\kappa$.

Proposition 4.15 (Decomposition of one rank)**.** *The action (X_r, M_r) divides a wreath product with the following factors, in the order listed:*

(i) *a reset action on J_r;*
(ii) *the finite transformation monoid $(\mathscr{C}, U_r)$ of block bookkeeping from Definition 4.13;*
(iii) *a reset action on X_r;*
(iv) *one regular action with resets $(H_{P_\kappa}, \mathcal{G}(H_{P_\kappa}))$ for each recurrent class κ, in any order;*
(v) *the direct product of $K_r - 1$ copies of the reset action $(\mathrm{Mat}_d(\mathbb{R}), \mathcal{M}(\mathrm{Mat}_d(\mathbb{R})))$.*

Proof. If $X_r = \{\varnothing\}$, the action is trivial and the assertion follows. Otherwise put $K = K_r$. As in Definition 2.2, consider the wreath product (Y, L) with

$$Y = J_r \times \mathscr{C} \times X_r \times \prod_\kappa H_{P_\kappa} \times \mathrm{Mat}_d(\mathbb{R})^{K-1},$$

where the product ranges over recurrent classes at rank r. By Definition 2.3, to show that (X_r, M_r) divides (Y, L), we must specify a submonoid $L_0 \le L$, an invariant subset $Y_0 \subseteq Y$, and compatible maps $q_1 \colon Y_0 \to X_r$ and $q_2 \colon L_0 \to M_r$.

Step 1. We specify the data stored in each coordinate of Y. Write its elements as

$$y = (A;\, t;\, B;\, (g_\kappa)_\kappa;\, Z_1, \dots, Z_{K-1}).$$

Starting from the initial point $y_* = (\varnothing;\, 0;\, \varnothing;\, (\mathrm{id}_{P_\kappa})_\kappa;\, 0, \dots, 0)$, the transformations below give these coordinates the following meaning:

- A stores the last matrix applied to the rank action.
- $t = 0$ means that the stored product is $\varnothing$. Otherwise $t = (k, \kappa)$ means that k is the total number of blocks in the sequence of nonempty inputs since the last reset to $\varnothing$, and κ labels only the last block.
- B stores the first matrix of the last block.
- If that block belongs to a recurrent class κ, then g_κ stores the group product in (4.12). The other group coordinates are not used to recover this block.
- $Z_1, \dots, Z_{k-1}$ store the matrix products of the completed blocks, in order. The remaining Z_j are not used.

Step 2. We define L_0. First, we construct maps $\widetilde{\theta}_E \colon Y \to Y$ for each $E \in J_r$. Let $\widetilde{\theta}_\varnothing = \mathrm{id}_Y$. Fix a matrix $E \in J_r$. For each tuple $y = (A;\, t;\, B;\, (g_\kappa);\, Z_1, \dots, Z_{K-1}) \in Y$, we specify

$$y\widetilde{\theta}_E = (A';\, t';\, B';\, (g'_\kappa)_\kappa;\, Z'_1, \dots, Z'_{K-1}) \in Y$$

by describing its coordinates; every coordinate not specified below stays unchanged. Set $A' = E$ and:

(a) If $\operatorname{rank} E < r$, or if $t \neq 0$ and $\operatorname{rank}(AE) < r$, put $t' = 0$.

(b) If $t = 0$ and $\operatorname{rank} E = r$, put

$$t' = (1, \kappa(E)), \qquad B' = E, \qquad g'_\nu = \mathrm{id}_{P_\nu} \quad (\nu \text{ recurrent}).$$

(c) If $t = (k, \kappa)$ with $\kappa \neq \circ$, $A, E \in \kappa$, and $\operatorname{rank}(AE) = r$, put

$$g'_\kappa = g_\kappa h(A, E).$$

(d) If $t = (k, \kappa)$ with $\operatorname{rank} E = \operatorname{rank}(AE) = r$ and either $\kappa = \circ$ or A, E do not both belong to κ, proceed as follows. Define $C : Y \to \mathrm{Mat}_d(\mathbb{R})$ by

$$C(y) = \begin{cases} B\beta_B g_\kappa \beta_A^{-1}, & \kappa \neq \circ,\ A, B \in \kappa, \\ B, & \kappa = \circ,\ B \neq \varnothing, \\ 0, & \text{otherwise.} \end{cases} \tag{4.16}$$

If $k < K$, put

$$t' = (k+1, \kappa(E)), \qquad B' = E, \qquad (g'_\nu)_\nu = (\mathrm{id}_{P_\nu})_\nu, \qquad Z'_k = C(y).$$

If $k = K$, use the same formulas for $t', B', (g'_\nu)_\nu$, with $k+1$ replaced by K, and leave all Z_j unchanged. We will show that case (d) with $k = K$ is never reached from y_*.

We now verify that $\widetilde{\theta}_E \in L$, checking the factors in the order listed in the proposition:

(i) On J_r, the map $A \mapsto E$ is constant.

(ii) On $\mathscr{C}$, the map is $\gamma_{A,E} \in U_r$, chosen using A.

(iii) On X_r, the map on B is the identity or the constant map with value E, chosen using A, t.

(iv) On each H_{P_ν}, the map is the identity, a right translation, or the constant map with value id_{P_ν}. The choice and the translating element depend only on A, t.

(v) On the matrix tuple, the map is the identity except in case (d) with $k < K$, when it is the element

$$(\mathrm{id}, \dots, \mathrm{id}, K_{C(y)}, \mathrm{id}, \dots, \mathrm{id}) \in \mathcal{M}(\mathrm{Mat}_d(\mathbb{R}))^{K-1},$$

with $K_{C(y)}$ in coordinate k. The case and the coordinate k are determined by A, t, E, and (4.16) shows that $C(y)$ depends only on $A, t, B, (g_\kappa)_\kappa$. Thus the transformation of the whole tuple is chosen using only the preceding factors.

Thus each factor transformation belongs to its stated monoid and is chosen using only preceding coordinates, as required by Definition 2.2. Set

$$L_0 = \langle \widetilde{\theta}_E : E \in J_r \rangle \leq L.$$

Step 3. We construct Y_0 and the surjection q_1. Let $Y_0 = y_* L_0$. This set is nonempty and L_0-invariant. Define

$$q_1 : Y_0 \longrightarrow X_r, \qquad q_1(y) = \begin{cases} \varnothing, & t = 0, \\ Z_1 \cdots Z_{k-1} C(y), & t = (k, \kappa). \end{cases}$$

To check that this formula takes values in X_r, we prove by induction on n that, for every $E_1, \dots, E_n \in J_r$,

$$q_1(y_*\widetilde{\theta}_{E_1} \cdots \widetilde{\theta}_{E_n}) = \varnothing \theta^{(r)}_{E_1} \cdots \theta^{(r)}_{E_n}.$$

The right-hand side belongs to X_r by definition of the maps $\theta^{(r)}_E$. We also verify the coordinate descriptions in Step 1: when $t = (k, \kappa)$, Z_j is the product of the jth block for $j < k$, and $C(y)$ is the product of the kth block.

For $n = 0$, the formula gives $q_1(y_*) = \varnothing$. For the induction step, write $y = y_*\widetilde{\theta}_{E_1} \cdots \widetilde{\theta}_{E_n}$ and consider $y\widetilde{\theta}_E$. If $E = \varnothing$, both transformations are identities. Suppose that E is a matrix. If $t = 0$ and $\operatorname{rank} E < r$, case (a) gives $t' = 0$, so $q_1(y\widetilde{\theta}_E) = \varnothing$. If $t = 0$ and $\operatorname{rank} E = r$, case (b) gives $A' = B' = E$, $t' = (1, \kappa(E))$ and identity group coordinates. Thus $C(y\widetilde{\theta}_E) = E$ by (4.16), and $q_1(y\widetilde{\theta}_E) = E$.

Now suppose $t \neq 0$, and let $Q = q_1(y)$ be the rank-r product given by the induction hypothesis. Its last matrix is A, so $VQ \subseteq VA$. Both spaces have dimension r, hence

$$VQ = VA, \qquad \operatorname{rank}(QE) = \operatorname{rank}(AE).$$

If this rank is less than r, case (a) gives $t' = 0$, as required by the definition of $\theta^{(r)}_E$. Otherwise appending E gives another rank-preserving sequence. If E belongs to the same recurrent class as its last block, case (c) updates that block's product by Proposition 4.11. In all other cases, (d) stores the old block product and starts (E). This sequence has at most K blocks by Lemma 4.9, so case (d) cannot require a $(K+1)$st block. Every Z_j used in q_1 has therefore been assigned the product of the corresponding block. Values left from an earlier sequence are not used. This proves the induction and, in particular, that q_1 takes values in X_r. It also gives the compatibility identity

$$q_1(y\widetilde{\theta}_E) = q_1(y)\theta^{(r)}_E \qquad (y \in Y_0,\ E \in J_r). \tag{4.17}$$

The map q_1 is surjective: $q_1(y_*) = \varnothing$, and for a rank-r matrix $A \in \mathcal{S}$, (b) gives $q_1(y_*\widetilde{\theta}_A) = A$.

Step 4. We construct the surjective homomorphism q_2. Define

$$q_2 \colon L_0 \longrightarrow M_r, \qquad q_2(\widetilde{\theta}_{E_1} \cdots \widetilde{\theta}_{E_n}) = \theta^{(r)}_{E_1} \cdots \theta^{(r)}_{E_n},$$

with the empty product sent to id_{X_r}. If two products on the left define the same transformation of Y, they agree on Y_0. Iterating (4.17) shows that the corresponding products on the right agree on $q_1(Y_0) = X_r$. Thus q_2 is well-defined. Concatenation shows that it is a monoid homomorphism, and it is surjective because its image contains every generator $\theta^{(r)}_E$ of M_r. Finally, iterating the same identity gives

$$q_1(yu) = q_1(y)q_2(u) \qquad (y \in Y_0,\ u \in L_0).$$

These are precisely the requirements of Definition 2.3. □

We collect here a technical result that will be needed in the implementation in Appendix A.3.

Corollary 4.18. *Suppose $\mathcal{S}$ is generated by finitely many matrices $(T_x)_{x\in\Sigma}$. Apply the constructions of Propositions 4.2 and 4.15 to a word $w \in \Sigma^*$, starting from their specified*

initial states. Every matrix stored in a factor (v) *of Proposition 4.15 is zero or T_v for a consecutive subword v of w. In particular, its maximum norm is at most $C^{|w|}$, where*

$$C = \max\{1, \max_{x \in \Sigma} \|T_x\|_{\mathrm{op}}\}$$

and $\|\cdot\|_{\mathrm{op}}$ is the operator norm induced by the maximum norm on $\mathbb{R}^d$.

Proof. Track words alongside the matrices in Proposition 4.2. On reading x, the input $E_d = T_x$ represents the one-letter word x. At rank $r \geq 1$, concatenate the words representing y_r and E_r. The rule (4.3) either stores this concatenation at rank r and passes the empty word, or passes it to rank $r-1$ and empties rank r. At rank zero, retain the concatenation. Thus the coordinate words, in order from rank zero to rank d, always concatenate to the prefix read. Every nonempty input at rank r empties all higher ranks, so the next such input represents exactly the letters read since then. The nonempty inputs at each rank therefore represent successive, disjoint, consecutive pieces of w.

In Step 3 of Proposition 4.15, a matrix coordinate is assigned the product of a block of consecutive nonempty rank inputs. It is therefore T_v for a consecutive subword v of w. All matrix coordinates start at zero, and a coordinate left unchanged retains an earlier such product. Finally, $\|T_v\|_\infty \leq \|T_v\|_{\mathrm{op}} \leq C^{|v|} \leq C^{|w|}$. □

Some constructions in this subsection, in particular the notions of blocks and recurrent sequences and blocks are inspired by [6, Section 6].

4.3. **Combining the decompositions.** To combine Propositions 4.2 and 4.15, we use that wreath products preserve division in each factor [8, Chapter XII, Section 2]. Thus each factor in Proposition 4.2 can be replaced by a wreath product that covers it.

Proof of Theorem 2.7. By Proposition 4.2,

$$(\mathcal{S}, \mathcal{S}) \mid (X_d, M_d) \wr \cdots \wr (X_0, M_0).$$

For each $1 \leq r \leq d$, replace (X_r, M_r) by the wreath product given in Proposition 4.15. The direct product in (v) divides the wreath product of its reset actions: take the submonoid with constant dependency functions and the identity map on their common state space. We may therefore replace that direct product by the individual reset factors. The rank-zero factor divides the reset action on X_0: its transformations are the identity and, when $0 \in \mathcal{S}$, the constant map with value 0. The properties above and transitivity of division therefore give a division of $(\mathcal{S}, \mathcal{S})$ into a finite wreath product of the listed factors.

The group factors are regular actions with resets of H_{P_κ}, where each P_κ is an eventual image. The finite factors have only subgroups of order at most two by Lemma 4.14, and all other factors are reset actions. These are the three types required by the theorem. □

To obtain an efficient transformer implementation of Theorem 2.7, we will further decompose the group-with-resets actions $(H, \mathcal{G}(H))$. The following result builds on the classical universal property of wreath product for groups [9], and the use of a reset factor in [2, Lemma 11].

Proposition 4.19 (Decomposition of a group with resets)**.** *Let H^0 be a normal subgroup of a group H, and put $\Phi = H/H^0$. Then*

$$(H, \mathcal{G}(H)) \mid (\Phi, \Phi) \wr (\Phi, \mathcal{M}(\Phi)) \wr (H^0, \mathcal{G}(H^0)).$$

Proof. Let (Y, L) be the wreath product in the statement, so that $Y = \Phi \times \Phi \times H^0$. Let $\pi\colon H \to \Phi$ be the quotient homomorphism, and let e be the identity of Φ. Choose representatives $m_c \in H$ with $\pi(m_c) = c$ for each $c \in \Phi$, taking $m_e = 1$. Let $Y_0 = Y$ and

$$q_1\colon Y \to H, \qquad q_1(c, b, u) = m_b^{-1} u m_c.$$

The map q_1 is surjective, since $q_1(\pi(h), e, h m_{\pi(h)}^{-1}) = h$ for every $h \in H$.

Consider the auxiliary map

$$a\colon \Phi \times H \to H^0, \qquad a(c, h) = m_c h m_{c\pi(h)}^{-1}.$$

This takes values in H^0 because $\pi(a(c,h)) = c\pi(h)(c\pi(h))^{-1} = e$.

To define L_0, we introduce for each $h \in H$ maps (corresponding to R_h and K_h) by

$$(c, b, u)\widetilde{R}_h = \big(c\pi(h),\ b,\ ua(c,h)\big), \qquad (c, b, u)\widetilde{K}_h = \big(c\pi(h),\ c,\ a(c,h)\big).$$

These belong to L, because each of the entries depend only on the preceding coordinates. Let $L_0 \le L$ be the submonoid generated by these transformations.

For $y = (c, b, u) \in Y$, we have

$$q_1(y\widetilde{R}_h) = m_b^{-1} u a(c,h) m_{c\pi(h)} = m_b^{-1} u m_c h = q_1(y) R_h,$$
$$q_1(y\widetilde{K}_h) = m_c^{-1} a(c,h) m_{c\pi(h)} = h = q_1(y) K_h.$$

These identities and the surjectivity of q_1 ensure that the assignments $\widetilde{R}_h \mapsto R_h$ and $\widetilde{K}_h \mapsto K_h$ extend to a well-defined surjective monoid homomorphism $q_2\colon L_0 \to \mathcal{G}(H)$, with $q_1(y\ell) = q_1(y) q_2(\ell)$. This proves the stated division. □

4.4. **Solvable linear groups.** By [2, Theorems 2 and 4], existence of exact transformer simulations of HMMs with finitely many beliefs is deeply connected with solvability of certain finite groups.

For arbitrary GHMMs, our cascade decomposition (Theorem 2.7) reduces the group computations to the (generally infinite) linear groups H_P. The Lie–Kolchin Theorem states that any *solvable* linear group separates into a triangular subgroup and a finite solvable quotient. Our implementation, based on Proposition 4.19, treats these two parts separately and also requires a polynomial description of the cosets, so we state a slight variation of this classical result.

Theorem 4.20 (Lie–Kolchin)**.** *Let $H \le \mathrm{GL}_r(\mathbb{C})$ be a solvable group. There is a normal subgroup $H^0 \trianglelefteq H$ such that*

(i) *all elements of H^0 are simultaneously upper triangular in one basis of $\mathbb{C}^r$;*
(ii) *the quotient $\Phi = H/H^0$ is finite and solvable;*
(iii) *each coset of H^0 is the set of elements of H satisfying a fixed finite collection of polynomial equations in the matrix entries.*

Proof. Let G be the Zariski closure of H in $\mathrm{GL}_r(\mathbb{C})$, the set of invertible matrices satisfying every polynomial equation that holds on H. It is an algebraic group, and remains solvable because the iterated commutator identities on H extend to the closure. Its identity component G° is normal (conjugation by $g \in G$ is a homeomorphism fixing the identity) and has finite index (G° cosets gG° correspond to connected components of G). By Lie–Kolchin, G° is upper triangular in one basis ([5, Chapter 1 and Theorem 16.30]). Set $H^0 = H \cap G^\circ$. Then H/H^0 embeds in the finite group G/G°, and is solvable as it is a quotient of H. Finally, since $hH^0 = H \cap hG^\circ$ and each each component hG° is cut out in $\mathrm{GL}_r(\mathbb{C})$ by finitely many polynomial equations, the same holds for hH^0. □

The polynomial equations in (iii) let the implementation determine the coset of a group element from its matrix entries. This explains the choice of subgroup H^0 in the proof; finite index and triangularity alone would not supply these tests.

4.5. **Triangular groups with resets.** Let F be a field and $H \leq \mathrm{GL}_r(F)$ a group of upper-triangular matrices. We compute the successive products in the regular action with resets $(H, \mathcal{G}(H))$ required by Proposition 4.19.

Definition 4.21. Fix $n \geq 1$, matrices $A_0, \dots, A_n \in H$, and reset indices $\mathcal{Z} \subseteq \{0, \dots, n\}$ with $0 \in \mathcal{Z}$. Starting at A_0, we apply $g \mapsto gA_i$ when $i \notin \mathcal{Z}$ and $g \mapsto A_i$ when $i \in \mathcal{Z}$. The successive group elements are

$$Y_i = \begin{cases} A_i, & i \in \mathcal{Z}, \\ Y_{i-1}A_i, & i \notin \mathcal{Z}. \end{cases}$$

Writing $s(i) = \max(\mathcal{Z} \cap \{0, \dots, i\})$ for the last reset, we have $Y_i = A_{s(i)}A_{s(i)+1} \cdots A_i$.

The *multiplier support* is

$$\Omega = \{(a, b) \colon 1 \leq a < b \leq r,\ (A_i)_{ab} \neq 0 \text{ for some } i \notin \mathcal{Z}\}.$$

Its column heights and maximum height are

$$h_\Omega(b) = \max(\{0\} \cup \{h_\Omega(a) + 1 \colon (a, b) \in \Omega\}), \qquad \tau(\Omega) = \max_{1 \leq b \leq r} h_\Omega(b).$$

Equivalently, $h_\Omega(b)$ is the largest ℓ for which there are indices $b_0 < \cdots < b_\ell = b$ with $(b_{j-1}, b_j) \in \Omega$ for $1 \leq j \leq \ell$. In particular, $\tau(\Omega) \leq r - 1$.

First replace each multiplier by its diagonal part, keeping each reset matrix in full. Since the matrices A_j are triangular and invertible, their diagonal entries are nonzero. For $0 \leq i \leq n$ and $1 \leq b \leq r$, put

$$q_{i,b} = \prod_{\substack{1 \leq j \leq i \\ j \notin \mathcal{Z}}} (A_j)_{bb},$$

with empty products equal to 1. The products with diagonal multipliers are the upper-triangular matrices B_i given, for $1 \leq a \leq b \leq r$, by

$$(B_i)_{ab} = (A_{s(i)})_{ab} \prod_{j=s(i)+1}^{i} (A_j)_{bb} = \frac{q_{i,b}}{q_{s(i),b}} (A_{s(i)})_{ab}. \tag{4.22}$$

Put $\mathbf{B} = (B_0, \dots, B_n)$ and $\mathbf{Y} = (Y_0, \dots, Y_n)$. To recover the contributions of the off-diagonal entries, define an affine operator $\mathcal{U}$ on sequences $\mathbf{X} = (X_0, \dots, X_n)$ of upper-triangular $r \times r$ matrices over F by

$$(\mathcal{U}(\mathbf{X})_i)_{ab} = (B_i)_{ab} + q_{i,b} \sum_{k=s(i)+1}^{i} \frac{1}{q_{k,b}} \sum_{\substack{a \le c < b \\ (c,b) \in \Omega}} (X_{k-1})_{ac} (A_k)_{cb}, \tag{4.23}$$

for $0 \le i \le n$ and $1 \le a \le b \le r$. At a reset the outer sum is empty, so $\mathcal{U}(\mathbf{X})_i = A_i$. The diagonal entries are always $(B_i)_{bb}$.

Proposition 4.24. *With the notation above, for every integer $t \ge \tau(\Omega)$,*

$$\mathcal{U}^t(\mathbf{B}) = \mathbf{Y}. \tag{4.25}$$

Proof. First we show that $\mathcal{U}(\mathbf{Y}) = \mathbf{Y}$. For $s(i) < k \le i$, matrix multiplication gives

$$\frac{(Y_k)_{ab}}{q_{k,b}} - \frac{(Y_{k-1})_{ab}}{q_{k-1,b}} = \frac{1}{q_{k,b}} \sum_{\substack{a \le c < b \\ (c,b) \in \Omega}} (Y_{k-1})_{ac} (A_k)_{cb},$$

since $q_{k,b} = q_{k-1,b}(A_k)_{bb}$. Summing from $s(i)+1$ to i and using $Y_{s(i)} = A_{s(i)}$ and (4.22) gives $Y_i = \mathcal{U}(\mathbf{Y})_i$. This also holds at resets, where the sum is empty.

We now prove by induction on $t \ge 0$ that

$$(\mathcal{U}^t(\mathbf{B})_i)_{ab} = (Y_i)_{ab} \qquad \text{whenever } h_\Omega(b) \le t, \quad 0 \le i \le n, \quad 1 \le a \le b \le r.$$

If $h_\Omega(b) = 0$, no pair (c, b) belongs to Ω, so the inner sum in (4.23) is empty. The fixed-point identity therefore gives $(Y_i)_{ab} = (B_i)_{ab}$, proving the case $t = 0$.

Suppose the claim holds for t and let $h_\Omega(b) \le t+1$. Each contributing column c satisfies $h_\Omega(c) < h_\Omega(b)$, hence $h_\Omega(c) \le t$. The induction hypothesis lets us replace the corresponding entries of $\mathcal{U}^t(\mathbf{B})$ by those of $\mathbf{Y}$ in (4.23), giving

$$(\mathcal{U}^{t+1}(\mathbf{B})_i)_{ab} = (\mathcal{U}(\mathbf{Y})_i)_{ab} = (Y_i)_{ab}.$$

Since every column has height at most $\tau(\Omega)$, this proves (4.25). □

5. Transformer architecture and simulation

This section gives an exact description of our transformers, then states the approximation and resource requirements for this simulation.

5.1. Layers.

Fix an integer $m \ge 1$, the *embedding dimension*. For each $n \ge 0$, the layers below act on sequences $Z = (z_0, \dots, z_n) \in (\mathbb{R}^m)^{n+1}$, with each z_i a column vector.

Definition 5.1 (Causal attention)**.** Let $H \ge 1$. An *H-headed causal attention block* is parameterized by affine maps

$$Q_h, K_h \colon \mathbb{R}^m \longrightarrow \mathbb{R}^{a_h}, \qquad V_h \colon \mathbb{R}^m \longrightarrow \mathbb{R}^{b_h},$$

and linear maps $O_h\colon \mathbb{R}^{b_h} \to \mathbb{R}^m$, where $1 \le h \le H$. These maps specify the h-th *attention head*; Q_h, K_h, V_h are its *query, key and value* maps, and O_h maps its output back to $\mathbb{R}^m$. For $0 \le t \le i \le n$, the weight that head h, at position i, assigns to position t is the softmax

$$\alpha_{it}^h(Z) = \frac{\exp\bigl(\langle Q_h(z_i), K_h(z_t)\rangle/\sqrt{a_h}\bigr)}{\sum_{s=0}^{i} \exp\bigl(\langle Q_h(z_i), K_h(z_s)\rangle/\sqrt{a_h}\bigr)}.$$

The associated *causal attention map* $A\colon (\mathbb{R}^m)^{n+1} \to (\mathbb{R}^m)^{n+1}$ is given by

$$A(Z)_i = \sum_{h=1}^{H} O_h \left(\sum_{t=0}^{i} \alpha_{it}^h(Z) V_h(z_t) \right).$$

Definition 5.2 (Feedforward map)**.** Let $k \ge 1$. A *ReLU feedforward block with k hidden layers* is parameterized by affine maps

$$\phi_j\colon \mathbb{R}^{w_{j-1}} \longrightarrow \mathbb{R}^{w_j} \qquad (1 \le j \le k+1),$$

where $w_0 = w_{k+1} = m$ and $w_1, \dots, w_k$ are the *hidden-layer widths*. These define the function

$$f\colon \mathbb{R}^m \longrightarrow \mathbb{R}^m, \qquad f = \phi_{k+1} \circ \mathrm{ReLU} \circ \phi_k \circ \cdots \circ \mathrm{ReLU} \circ \phi_1,$$

where $\mathrm{ReLU}(t) = \max\{0, t\}$ is applied coordinatewise. The associated *feedforward map* is

$$F\colon (\mathbb{R}^m)^{n+1} \longrightarrow (\mathbb{R}^m)^{n+1}, \qquad F(Z) = (f(z_0), \dots, f(z_n)).$$

We call $\max\{w_1, \dots, w_k\}$ its *hidden width*.

Definition 5.3 (Layer normalization)**.** Fix a real number $\delta > 0$. The *layer normalization map* is parameterized by vectors $\gamma, \beta \in \mathbb{R}^m$, called its *gain* and *bias*, and defined by

$$\mathrm{LN}_{\gamma,\beta}\colon \mathbb{R}^m \longrightarrow \mathbb{R}^m, \quad z \longmapsto \gamma \odot \frac{z - \bar{z}\mathbf{1}}{\sqrt{m^{-1}\|z - \bar{z}\mathbf{1}\|_2^2 + \delta}} + \beta, \qquad \bar{z} = \frac{1}{m}\sum_{a=1}^{m} z_a,$$

where $\odot$ denotes coordinatewise multiplication. The associated map on sequences is

$$\mathcal{N}_{\gamma,\beta}(Z) = (\mathrm{LN}_{\gamma,\beta}(z_0), \dots, \mathrm{LN}_{\gamma,\beta}(z_n)).$$

Definition 5.4 (Transformer layer)**.** A *transformer layer* consists on a causal attention map A, a feedforward map F, and two layer normalization maps $\mathcal{N}_A, \mathcal{N}_F$. It is the map $B\colon (\mathbb{R}^m)^{n+1} \to (\mathbb{R}^m)^{n+1}$ given by

$$\widetilde{Z} = Z + A(\mathcal{N}_A(Z)), \qquad B(Z) = \widetilde{Z} + F(\mathcal{N}_F(\widetilde{Z})).$$

The two additions are the *residual connections*. Each unnormalized vector enters only through its residual addition.

5.2. **Transformers from words to outputs.**

Definition 5.5 (Full transformer)**.** Fix an alphabet Σ, a horizon $N \geq 1$, a nonnegative integer L, and positive integers m, d. Choose a symbol $\star \notin \Sigma$. A *transformer* of depth L, embedding dimension m, and output dimension d consists of:

(i) vectors $p_0, \dots, p_N \in \mathbb{R}^m$, called the *positional encoding*;
(ii) an *input embedding* $e\colon \Sigma \sqcup \{\star\} \to \mathbb{R}^m$ which induces

$$E_N\colon \Sigma^N \longrightarrow (\mathbb{R}^m)^{N+1}, \qquad x_1 \cdots x_N \longmapsto (e(\star) + p_0, e(x_1) + p_1, \dots, e(x_N) + p_N),$$

(iii) L layers $B_1, \dots, B_L$ as in Definition 5.4;
(iv) a final normalization $\mathrm{LN}_{\gamma_{\mathrm{out}}, \beta_{\mathrm{out}}}\colon \mathbb{R}^m \to \mathbb{R}^m$;
(v) and an affine *output map* $\rho\colon \mathbb{R}^m \to \mathbb{R}^d$, which induces

$$R_N\colon (\mathbb{R}^m)^{N+1} \longrightarrow (\mathbb{R}^d)^N, \qquad (z_0, \dots, z_N) \longmapsto (\rho(z_1), \dots, \rho(z_N))$$

The *full transformer* $\mathcal{T}_{\mathrm{full}}\colon \Sigma^N \to (\mathbb{R}^d)^N$ is the composition

$$\mathcal{T}_{\mathrm{full}} = R_N \circ \mathcal{N}_{\mathrm{out}} \circ B_L \circ \cdots \circ B_1 \circ E_N.$$

For $L = 0$, the composition of layers is the identity.

The alphabet Σ is either finite, or a subset of $\mathbb{R}^D$ with $e|_\Sigma$ the restriction of an affine map.

We allow attention and feedforward blocks, normalization maps (including the final one), and residual connections to be omitted.

5.2.1. *Complexity measures.* Following [2, Appendix A.4], we use the following complexity measures for a transformer as in Definition 5.5. For layer ℓ, write H_ℓ for its number of heads, $a_{\ell h}, b_{\ell h}$ for their dimensions, and $k_\ell, w_{\ell 1}, \dots, w_{\ell k_\ell}$ for its number of hidden layers and their widths.

(i) The *horizon* is N and the *embedding dimension* is m.
(ii) The *transformer depth* is L, and the *feedforward depth* is $K = \max_\ell k_\ell$.
(iii) The *number of heads* is the total $\sum_{\ell=1}^{L} H_\ell$, unless a particular layer is specified.
(iv) The *attention width* and *feedforward width* are

$$W_{\mathrm{attn}} = \max_\ell \left\{ m, \sum_{h=1}^{H_\ell} a_{\ell h}, \sum_{h=1}^{H_\ell} b_{\ell h} \right\}, \qquad W_{\mathrm{ff}} = \max_{\ell, j} w_{\ell j}.$$

(v) The *parameter count* is the total number of scalar entries in the matrices and vectors specifying the transformer. The *parameter magnitude* is their largest absolute value.

Empty sums and maxima in these complexity measures are zero.

5.3. **Simulation of successive updates.**

Definition 5.6 (Simulation)**.** Let $\mathcal{A} = (Q, \Sigma, (f_x)_{x \in \Sigma})$ be a semiautomaton and fix $q_0 \in Q$. For $N \geq 1$, define the map of successive values by

$$\mathcal{A}_{N, q_0}\colon \Sigma^N \longrightarrow Q^N, \qquad x_1 \cdots x_N \longmapsto (q_1, \dots, q_N), \qquad q_i = f_{x_i}(q_{i-1}) \quad (1 \leq i \leq N).$$

(i) A map $\Psi\colon \Sigma^N \to Q^N$ *simulates* $\mathcal{A}_{N,q_0}$ exactly if $\Psi = \mathcal{A}_{N,q_0}$.
(ii) If $Q \subseteq \mathbb{R}^d$ and $\varepsilon \geq 0$, a map $\Psi\colon \Sigma^N \to (\mathbb{R}^d)^N$ *ε-simulates* $\mathcal{A}_{N,q_0}$ if

$$\sup_{w\in\Sigma^N} \max_{1\leq i\leq N} \bigl\|\Psi(w)_i - \mathcal{A}_{N,q_0}(w)_i\bigr\|_\infty \leq \varepsilon.$$

(iii) Suppose $Q \subseteq \mathbb{R}^d$ and $\Sigma \subseteq W \subseteq \mathbb{R}^D$. For $\varepsilon, \nu \geq 0$, a map $\Psi\colon W^N \to (\mathbb{R}^d)^N$ *ε-simulates* $\mathcal{A}_{N,q_0}$ *at tolerance* ν if for all $w \in \Sigma^N$ and $\widehat{w} \in W^N$,

$$\|\widehat{w} - w\|_\infty \leq \nu \quad\Longrightarrow\quad \max_{1\leq i\leq N} \bigl\|\Psi(\widehat{w})_i - \mathcal{A}_{N,q_0}(w)_i\bigr\|_\infty \leq \varepsilon.$$

A transformer simulates these updates if its output map $\mathcal{T}_{\text{full}}$ satisfies the corresponding condition. Simulation up to horizon N requires the condition at every length $1 \leq n \leq N$.

For the belief semiautomaton, we apply this definition using the numerical representation $b \mapsto (1, b)$ and $\bot \mapsto (0, 0)$.

Definition 5.7 (Approximation)**.** Let $X \subseteq W \subseteq \mathbb{R}^k$ and $F\colon X \to \mathbb{R}^d$. For $\varepsilon, \nu \geq 0$, a function $\Psi\colon W \to \mathbb{R}^d$ *ε-approximates F at tolerance ν* if, for every $x \in X$ and $\widehat{x} \in W$,

$$\|\widehat{x} - x\|_\infty \leq \nu \quad\Longrightarrow\quad \|\Psi(\widehat{x}) - F(x)\|_\infty \leq \varepsilon.$$

6. Transformer simulation of the building blocks

The decompositions in Theorems 2.7 and 4.20 reduce the simulation of a solvable GHMM to certain *building blocks:* finite solvable groups, reset actions, and triangular group actions with resets. The finite factors are covered by [2]. Here we deal with the infinite ones.

6.1. **Triangular groups.** To simulate a triangular group with resets, we use Proposition 4.24 and the notation it introduces. Each application of $\mathcal{U}$ combines matrix entries from earlier positions; we explain how attention approximates these sums in one layer.

Let $e_i = |\mathcal{Z} \cap \{1, \dots, i\}|$ count the resets up to position i. Then $s(i) < j \leq i$ is equivalent to $j \leq i$, $e_j = e_i$, and $j \notin \mathcal{Z}$. For $(c, b) \in \Omega$ and $a \leq c$, the corresponding contribution to (4.23) can therefore be written as

$$q_{i,b} \sum_{\substack{1\leq j\leq i,\ j\notin\mathcal{Z}\\ e_j=e_i}} \frac{(A_j)_{cb}}{q_{j,b}} (X_{j-1})_{ac}.$$

Expanding the three factors into real and imaginary parts and rescaling reduces this to real sums $f_i \sum_{1\leq j\leq i,\, e_j=e_i} g_j v_j$, with $f_i, g_j \in [-1, 1]$ and bounded v_j, set to zero at resets. To obtain the possibly negative products $f_i g_j$ from positive attention weights, choose $C \geq 0$ and combine four heads with scores

$$\log(2 + sf_i) + \log(2 + tg_j) - C(e_i - e_j)^2, \qquad s, t \in \{-1, 1\}.$$

Once the logarithms and squared reset counts have been computed, these scores are query–key inner products. Their exponentials are $(2 + sf_i)(2 + tg_j)e^{-C(e_i-e_j)^2}$, with a factor at most e^{-C} suppressing contributions from earlier reset intervals.

The identity below combines the four numerators to give $f_i g_j$; different softmax denominators would spoil this cancellation. Give BOS score $\log E$ and value zero, where $E \geq 1$.

Since ordinary positions contribute at most $9N$, each denominator lies in $[E, E + 9N]$. Choosing $E \gg N$ makes them approximately equal, allowing us to combine the heads with coefficients $st/4$ using

$$\frac{1}{4} \sum_{s,t \in \{-1,1\}} st\,(2 + sf_i)(2 + tg_j) = f_i g_j.$$

Thus the combined output approximates the required sum multiplied by E^{-1}, with errors controlled by N/E and e^{-C}. The factor E^{-1} is absorbed into the scale of the stored matrices.

Four layers prepare $\mathbf{B}$ and the coefficients of $\mathcal{U}$, using the numerical constructions in Appendix A. Each subsequent layer approximates one application of $\mathcal{U}$, computing current and preceding values together for the next iteration. By Proposition 4.24, $\tau(\Omega)$ iterations suffice, giving depth $4 + \tau(\Omega)$ independently of the word length. Lemma B.1 gives the approximation and resource bounds, including errors in the supplied matrices.

Under the hypotheses of Lemma A.2, rational functions of supplied numerical coordinates admit one-layer approximation with fixed embedding dimension and the head, tolerance and parameter bounds stated there. Lemma A.4 copies arbitrary supplied real reset values exactly in two layers, without amplifying errors in the supplied values when the reset indicators are exact. Together with the partial sums of Lemma A.1, these constructions supply the four preparation layers already counted above.

7. Composition and belief recovery

We now connect the simulations of the factors to obtain a transformer whose inputs are the original letters and whose outputs are their successive beliefs.

7.1. **Assembling the rank computations.** Start at rank d, where each letter supplies its matrix T_x. The rule in (4.3) tells us what to pass to the next rank: if multiplying the stored matrix by the incoming matrix lowers the rank, pass that product down and reset the current state to $\varnothing$; otherwise retain the product and pass $\varnothing$. We compute all positions at one rank before proceeding to the next, in the order $d, d-1, \ldots, 0$.

Within each rank, Proposition 4.15 tells us how to assemble the factors. The finite factors determine when blocks begin and end. Reset actions retain their endpoints and completed products, while groups with resets compute the products within recurrent blocks. Each group is computed through its finite quotient and triangular subgroup, using Proposition 4.19 and the simulations of Section 6. Recovering both current and preceding values lets us store a block when it ends and compute the product passed to the next rank. That rank receives the matrices actually computed by the preceding network; Proposition D.2 includes tolerance for their errors.

7.2. **Controlling the approximation.** We choose accuracies backward from the final output, requiring each rank's error to lie within the next computation's tolerance. This requires quantitative control of divisions and of the tests deciding rank drops and changes of block. A nonzero denominator or test can be very small; finiteness of the words up to horizon N gives no useful bound on how much precision is needed.

This is where we use Proposition C.1. We make one common replacement of the numerical constants throughout the calculation, preserving the polynomial equalities and sign conditions that determine its choices, support and recovery identities. For an integer $Q \geq 2$, we also preserve comparisons placing each normalized belief coordinate in an interval of length $1/Q$. The final beliefs therefore change by at most $1/Q$, although intermediate matrices may change substantially. The replacement can be chosen so that magnitudes and inverses of nonzero quantities are bounded by 2^B, with B polynomial in $N + \log Q$. This makes the required precision compatible with polynomial resources. The replacement is only used up to horizon N; its precise guarantees are in Appendix C.

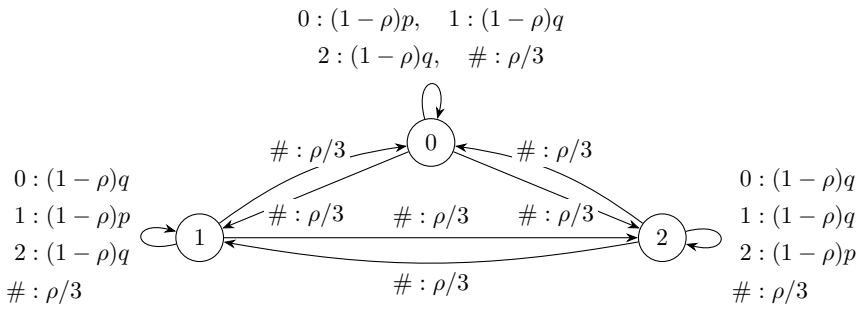
7.3. **Recovering the beliefs.** In exact arithmetic, Proposition 4.2 recovers each word product from the rank states, and Theorem 2.10 converts it to a belief by normalization. We apply these same formulas to the specialized constants and split the error allowance between specialization and transformer approximation. The preserved support and separation from zero let the final calculation distinguish impossible words exactly and return the sink.

The number of ranks, finite factors and triangular iterations depends only on the process. Thus composing their simulations keeps depth and embedding dimension independent of the horizon and accuracy. Proposition D.4 gives the resulting simulation and the resource bounds of Theorem 3.1.

8. A reset topic process

We apply our decomposition to a three-topic process with resets. Its diagonal group computation gives a concrete test of how trained transformers accumulate evidence and remove it at a reset.

8.1. **The process.** Fix $0 < \rho < 1$ and $1/3 < p < 1$, and put $q = (1-p)/2$. Take $\Sigma = \{0, 1, 2, \#\}$ and $V = \mathbb{R}^{1\times 3}$. The initial row and readout column are $\mu = (1/3, 1/3, 1/3)$ and $\ell = \mathbf{1} = (1, 1, 1)^\top$. The hidden-state graph is



The observation matrices are

$$T_0 = (1-\rho)\,\mathrm{diag}(p, q, q), \qquad T_1 = (1-\rho)\,\mathrm{diag}(q, p, q),$$
$$T_2 = (1-\rho)\,\mathrm{diag}(q, q, p), \qquad R = T_\# = \rho \mathbf{1}\mu. \tag{8.1}$$

Intuitively, at each step, this process:

- With probability $1-\rho$, keeps the current topic and emits its symbol with probability p, or each other symbol with probability q.
- With probability ρ, emits # and independently redraws the topic from μ.

One can see that all words have positive probability and that this realization is minimal.

8.2. **The rank decomposition.** Consider the submonoid $\mathcal{S} = \langle T_0, T_1, T_2, R\rangle \subseteq \mathrm{Mat}_3(\mathbb{R})$. Products of T_0, T_1, T_2 are invertible, while every product containing R has rank one. Thus, by Proposition 4.2, the regular action of $\mathcal{S}$ divides the wreath product of two factors:

$$(\mathcal{S}, \mathcal{S}) \mid (X_3, M_3) \wr (X_1, M_1).$$

To illustrate this division, we show how it organizes the product T_w for $w = 01\#20\#010$.

Reading 01 forms the rank-three product T_0T_1. The first # makes T_0T_1R rank one, so this completed product is retained at rank one. Reading 20 starts a new rank-three product T_2T_0; the second # produces the next rank-one factor T_2T_0R. The remaining 010 forms $T_0T_1T_0$ at rank three. Thus the decomposition gives

$$T_w = \underbrace{T_0T_1RT_2T_0R}_{\text{rank one}}\ \underbrace{T_0T_1T_0}_{\text{rank three}}.$$

At the end, the X_1-coordinate is $T_0T_1RT_2T_0R$, and the X_3-coordinate is $T_0T_1T_0$. Proposition 4.15 further decomposes each rank using stabilizer groups of eventual images.

8.3. **The eventual images.** For $x \in \{0,1,2\}^*$, we have

$$RT_xR = \sigma(T_x)R, \qquad \text{where } \sigma(T_x) = \rho\mu T_x\mathbf{1} > 0. \tag{8.2}$$

Repeated use of this identity shows that every product containing R has the form $A = cT_x\mathbf{1}\mu T_y$, with $c > 0$ and $x, y \in \{0,1,2\}^*$. It has rank one and image $VA = \mathrm{span}(\mu T_y)$.

The eventual images fall into two recurrent equivalence classes:

Rank three. V alone.

Rank one. The lines $\mathrm{span}(\mu T_y)$, with $y \in \{0,1,2\}^*$, including $P = \mathrm{span}(\mu) = VR$ when y is empty. They belong to the same recurrent class.

8.4. **The stabilizer groups.** Stabilizers within each recurrent class are conjugate (Proposition 4.11), so we choose V and P as representatives.

Rank three. A matrix in $\mathcal{S}$ preserves V if and only if it is invertible. These are precisely the products of T_0, T_1, T_2. Using that diagonal matrices commute, Definition 2.5 gives

$$H_V = \{T_0^{m_0}T_1^{m_1}T_2^{m_2} : (m_0, m_1, m_2) \in \mathbb{Z}^3\}.$$

Its action on V is right multiplication by the diagonal matrices

$$T_0^{m_0}T_1^{m_1}T_2^{m_2} = ((1-\rho)q)^{m_0+m_1+m_2}\,\mathrm{diag}\big((p/q)^{m_0}, (p/q)^{m_1}, (p/q)^{m_2}\big).$$

One can see that the generators are multiplicatively independent. Hence

$$H_V \xrightarrow{\sim} \mathbb{Z}^3, \qquad T_0^{m_0}T_1^{m_1}T_2^{m_2} \longmapsto (m_0, m_1, m_2).$$

Rank one. If $A \in \mathcal{S}$ satisfies $PA = P$, then $\mu A = c\mu$ for a unique $c > 0$: P is one-dimensional, μ is positive and A is nonnegative. Identifying the scalar map $c\,\mathrm{id}_P$ with its

multiplier c embeds H_P into $\mathbb{R}^{\times}_{>0}$, so H_P is abelian. Under this identification, $(T_xR)|_P$ corresponds to the scalar $\sigma(T_x)$ defined in (8.2), for every ordinary word x.

Here multiplication in the stabilizer groups is elementary: H_V consists of diagonal matrices and H_P acts by positive scalars. In general, the stabilizer groups need not be abelian. Under solvability, our construction approximates multiplication with resets in constant transformer depth.

8.5. **Belief recovery.** The belief formula $b_w = \mu T_w/(\mu T_w \mathbf{1})$ simplifies for this process. Let y be the suffix word after the last reset of w, or w itself if there is no reset. When $w = x\#y$, the identity $R = \rho\mathbf{1}\mu$ gives $\mu T_w = \rho(\mu T_x \mathbf{1})\mu T_y$. The positive scalar contributed by the rank-one component cancels in the normalization, giving

$$b_w = \frac{\mu T_y}{\mu T_y \mathbf{1}}. \tag{8.3}$$

On cascade states reached from the empty state by reading observation symbols, the X_3-coordinate recovers T_y, with $\varnothing$ interpreted as I. Thus (8.3) shows that belief recovery factors through the projection onto X_3. The X_1-coordinate can be omitted for belief recovery, although recovering the full matrix T_w still requires it.

For our word $w = 01\#20\#010$, the suffix is $y = 010$, and hence

$$b_w = \frac{\mu T_0 T_1 T_0}{\mu T_0 T_1 T_0 \mathbf{1}} = \frac{(p^2, pq, q^2)}{p^2 + pq + q^2}.$$

8.6. **Group actions in trained transformers.** Fix an observed word $w = w_1 \cdots w_N$. For $1 \le t \le N$, let $C_a(t)$ count occurrences of a in $w_1 \cdots w_t$, and let $s(t)$ be the position of the last # in this prefix, or 0 if there is none. Set $C_a(0) = 0$ and $y = w_{s(t)+1} \cdots w_t$. Representing the empty rank-three product by I, the diagonal construction (4.22) applies with each reset setting this product to I:

$$T_y = \underbrace{\prod_{a=0}^{2} T_a^{C_a(t)}}_{\substack{\text{ordinary symbols} \\ \text{up to } t}} \underbrace{\left(\prod_{a=0}^{2} T_a^{C_a(s(t))}\right)^{-1}}_{\substack{\text{inverse of their product} \\ \text{up to } s(t)}}.$$

Let n_a be the number of occurrences of symbol a in y. In the exponent coordinates $H_V \cong \mathbb{Z}^3$, the displayed computation becomes

$$n_a = C_a(t) - C_a(s(t)), \qquad a = 0, 1, 2. \tag{8.4}$$

We test whether next-token training learns this particular computation.

The current belief b gives next-token probabilities $(1-\rho)[pb_a + q(1-b_a)]$ for ordinary symbols a, and simply ρ for the symbol #. Write $n = (n_0, n_1, n_2)$ for the suffix counts and $Q(n)$ for the distribution conditional on the next symbol being ordinary. By (8.3),

$$Q_a(n) = q + (p-q)b_a = q + (p-q)\frac{(p/q)^{n_a}}{\sum_{j=0}^{2}(p/q)^{n_j}}, \qquad a = 0, 1, 2. \tag{8.5}$$

8.7. **Training.** We trained transformers by next-token prediction on sequences generated by this process, using two random seeds at each $p \in \{0.45, 0.40\}$ with $\rho = 0.01$. Each transformer has two layers, embedding dimension 64, four heads per layer, GELU width 256, pre-layer and final LayerNorm, and no positional encoding. Training used 30,000 updates, batch size 32 and context length 256. On 4,096 fresh length-256 documents per source, mean total-variation error in the full four-symbol prediction was 0.003–0.004.

8.8. **Interventions.** To test (8.4), we vary the counts before the last reset while holding the belief fixed. We pair a base history of length t with a donor obtained by resampling those observations from the process, conditional on the reset positions. We require $0 < s(t) < t$. Both histories retain the same suffix and reset positions, hence the same suffix counts n and belief.

Let C_a and C'_a be the count functions for the base and donor, respectively. Define $\delta = (\delta_0, \delta_1, \delta_2)$ by

$$\delta_a := C'_a(t) - C_a(t) = C'_a(s(t)) - C_a(s(t)), \qquad a = 0, 1, 2.$$

The equality holds because the suffixes agree. Thus δ_a is the change in the number of symbols a before the reset, from base to donor. For example, base 01#20 and donor 02#20 give $\delta = (0, -1, 1)$: one fewer 1 and one more 2.

We run the model on both histories and save the activations after its first layer. For each pair, we retain the unmodified base prediction and perform three separate interventions, each starting from the base:

(i) replace the activation at the current position t;
(ii) replace the activation at the last reset position $s(t)$;
(iii) replace both activations.

Each replacement copies the donor activation at the same position. We then recompute the second layer and the next-token prediction.

If the model uses these activations for the two count vectors in (8.4), the first intervention adds δ to the suffix counts, the second subtracts δ, and the third cancels the two changes. The predicted outputs are

$$Q(n) \longmapsto \begin{cases} Q(n+\delta), & \text{current position only,} \\ Q(n-\delta), & \text{last reset only,} \\ Q(n), & \text{both positions.} \end{cases} \tag{8.6}$$

Here (8.5) defines Q on all integer triples, including those with negative entries.

We applied all three interventions to 328 base–donor pairs per model, selected without consulting model outputs. These paired tests cover only changes with $\sum_a \delta_a = 0$; individual generator and inverse actions are tested separately below.

Results. The two single-position replacements approximately produce the probability changes prescribed by the group action (Figure 1). We quantify the agreement by the summed ℓ^1 discrepancy between observed and prescribed changes, divided by the summed

ℓ^1 size of the prescribed changes. Exact agreement gives zero. This relative error was 0.15–0.26 (Table 1).[1]

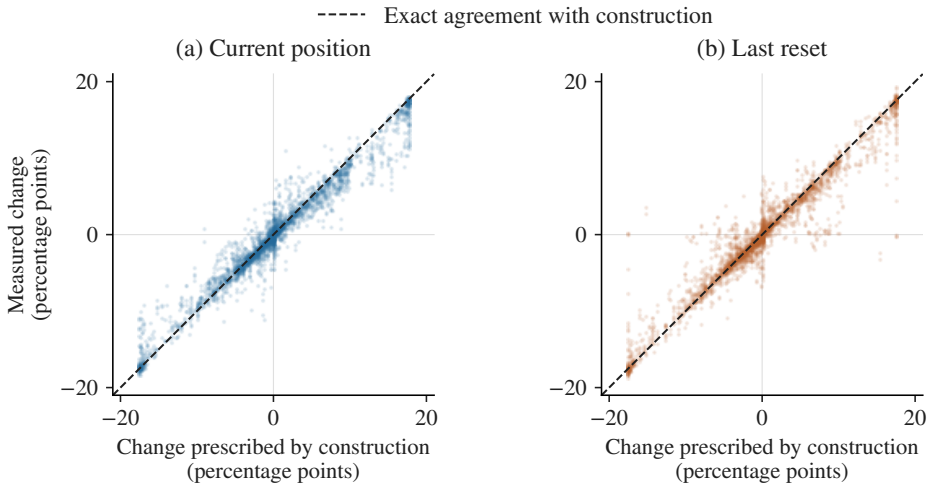


FIGURE 1. Prescribed and measured effects of the current and reset replacements. Each point is the change in one conditional ordinary-symbol probability for one history pair and one model; both panels include all 328 pairs from each of the four models. The dashed diagonal is exact agreement with (8.6).

Learning the actions. To measure how this agreement develops during training, we use separate controlled tests at $p = 0.45$. Without positional encoding, a first-layer activation depends only on its token and prefix counts. We regenerate the current or reset activation after increasing or decreasing one ordinary-symbol count by one, testing the actions of $T_a^{\pm 1}$, for $a = 0, 1, 2$. We measure their effects both immediately and after ordinary continuations.[2] Figure 2 shows that the relative action error falls from about 1 at initialization to 0.15–0.18 after training across the two seeds.

[1]These action errors include cases with theoretical total-variation change at least 0.002.

[2]For current edits, the same count shift is applied when computing first-layer activations at continuation positions. Reset edits change only the activation at the original reset position. Model weights are fixed.

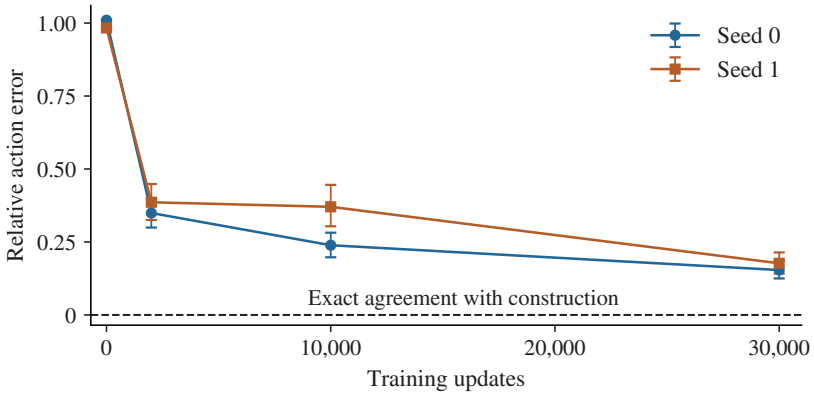


FIGURE 2. Relative action error at all four saved checkpoints, for both seeds at $p = 0.45$. The zero line is exact agreement with the prescribed actions. Errors are pooled over the immediate prediction and ten ordinary continuations of each of 40 histories from the exploratory sample; bars are 95% bootstrap intervals over these histories. This is a separate sample from Figure 1.

Cancellation. The joint replacement tests whether the two contributions cancel. Let $\widehat{Q}$ be the unmodified model's distribution conditioned on an ordinary next symbol, and write $\widehat{Q}^c, \widehat{Q}^r, \widehat{Q}^{cr}$ for the current, reset and joint replacements. An exact implementation would satisfy $\widehat{Q}^{cr} = \widehat{Q}$. We compare the residual joint effect with the sum of the separate effects:

$$\frac{\sum_i \mathrm{TV}(\widehat{Q}_i^{cr}, \widehat{Q}_i)}{\sum_i [\mathrm{TV}(\widehat{Q}_i^{c}, \widehat{Q}_i) + \mathrm{TV}(\widehat{Q}_i^{r}, \widehat{Q}_i)]}. \tag{8.7}$$

Exact cancellation gives zero. In the trained models, the joint replacement produces 8–12% of the sum of the separate output effects (Table 1).

		Relative action error (%)		Cancellation ratio (%)
p	Seed	Current only	Reset only	Joint
0.45	0	17.5 [15.1, 20.3]	15.4 [13.0, 18.3]	9.8 [8.3, 11.6]
0.45	1	18.3 [16.0, 20.8]	16.1 [13.4, 19.2]	11.6 [9.8, 13.5]
0.40	0	18.3 [16.5, 20.3]	24.3 [20.6, 28.4]	10.0 [8.3, 11.9]
0.40	1	20.2 [18.0, 22.7]	26.1 [22.4, 30.2]	8.2 [7.1, 9.6]

TABLE 1. Intervention results for each model, in percentages, with 95% intervals in brackets. Action errors use the stated theoretical effect threshold; cancellation uses all 328 pairs and the distinct normalization in (8.7). Intervals use 2,000 whole-document bootstrap resamples, keeping prefixes together, and quantify test-sample uncertainty for each fixed model.

Joint cancellation does not imply complete independence from earlier observations. We therefore also compare the unmodified model's predictions on the same 328 base–donor pairs. Equation (8.3) prescribes identical predictions; the full next-token distributions differ by mean total variation 0.0045–0.0062. The reset is therefore approximate even on unmodified inputs.

These tests support a learned approximation to (8.4): predictions respond to the current and reset contributions approximately as prescribed by the group action, while their common shift largely cancels.

Appendix A. Numerical calculations and exact copies

A.1. **Partial sums.** We use the partial-sum construction adapted from [2, Lemma 6].

Lemma A.1 (Partial sums)**.** *Let $D, N \geq 1$. The map*

$$(\mathbb{R}^D)^N \longrightarrow (\mathbb{R}^D)^N, \qquad (x_1, \ldots, x_N) \longmapsto (S_1, \ldots, S_N),$$

where $S_i = \sum_{j=1}^{i} x_j$, is computed exactly by a transformer of depth one and embedding dimension $D+2$. It uses one head of query and key dimension one and value dimension D. Its parameter count is $O(DN + D^2)$ and its parameter magnitude is at most $N+1$.

[Lean reference: partial sums.]

A.2. **Calculations at each position.** The following result applies to rational approximations of the logarithms and exponentials used in Section 6.1. For $f\colon K \to \mathbb{R}^d$, write $f^{\times N}(x_1, \ldots, x_N) = (f(x_1), \ldots, f(x_N))$.

Lemma A.2 (Approximation at each position)**.** *Let $k, d \geq 1$ and $f\colon K \to [-1,1]^d$, where $K \subseteq [-1,1]^k$. Fix $n, h \geq 1$ and $0 < \varepsilon \leq 1/8$. Suppose polynomials $P_0, \ldots, P_d$ of degree at most n and coefficient magnitude at most 2^h satisfy*

$$|P_0| \geq 2^{-h}, \qquad \left\| f - \frac{(P_1, \ldots, P_d)}{P_0} \right\|_\infty \leq \varepsilon/8 \quad \text{on } K.$$

For every $N \geq 1$, a transformer on $(\mathbb{R}^k)^N$ ε-approximates $f^{\times N}$ at tolerance $\nu > 0$, with depth one, embedding dimension $k+4d+5$, feedforward depth one and $O_{k,d}((n(1+\log(1/\varepsilon)))^k)$ heads. Integer-valued output coordinates are exact, and $\log(1/\nu) = O_{k,d}(h + \log(n+1) + \log(1/\varepsilon))$.

There is $\delta_0 > 0$ such that the same parameters and tolerance work for every common normalization stabilizer $0 < \delta \leq \delta_0$.

If f extends to the closed a-neighborhood of K, with the same bounds and Lipschitz constant L_f, one may take

$$\nu = \min\{1/2, a/2, \varepsilon/[8(L_f+1)]\}, \qquad 0 < a \leq 1. \tag{A.3}$$

[Lean references: local approximation and input tolerance.]

For an integer $D \geq 0$ and $z \in [-1,1]^D$, the same conclusion holds for $(x, z) \mapsto (f(x), z)$, with embedding dimension $k+4d+5+5D$ and $O_{k,d,D}((n(1+\log(1/\varepsilon)))^k(1+\log(1/\varepsilon)))$ heads. The tolerance bounds are unchanged, with constants depending also on D. If $C \subseteq \mathbb{Z}$ is fixed and finite, and every $f_c\colon K \to [-1,1]^d$ satisfies the lemma's hypotheses with common n, h, these bounds also apply to

$$(x, z, c) \longmapsto (f_c(x), z, c), \qquad c \in C,$$

with $O(1)$ additional embedding coordinates and constants depending on C. Labels are supplied and returned exactly; the tolerance applies to the numerical coordinates. With H heads, the parameter count is $O_{k,d,D,C}(N+H)$; parameter magnitude and $\log(1/\delta_0)$ are bounded by

$$O_{k,d,D,C}\big(\log(N+1) + n(1+\log(1/\varepsilon))^2(h+n+1)\big).$$

[Lean references: retained coordinates and finite labels.]

Precomposition by a fixed-dimensional affine map preserves the head count. For coefficients of magnitude at most 2^a, replace h by $h + O(na)$ in the magnitude and stabilizer bounds. Inputs in $[-M, M]^k$ are reduced to the unit cube by replacing h with $h + n \log_2 M$; outputs bounded by M use accuracy ε/M before rescaling, where $M \geq 1$. Integer outputs in $[-M, M]$ require feedforward width $O(M)$. *[Lean references: affine precomposition and rescaling.]*

A.3. **Exact resets.**

Lemma A.4 (Exact simulation of reset actions)**.** *Let $X \subseteq \mathbb{R}^D$ be non-empty and $m \geq 1$. Let $\mathcal{A}$ be the semiautomaton of the direct product $(X^m, \mathcal{M}(X)^m)$. Represent the identity by $(0, 0)$ and a reset to $u \in X$ by $(1, u)$, with one such pair for each factor of $\mathcal{M}(X)^m$.*

For every $q_0 \in X^m$ and $N \geq 1$, a transformer exactly simulates $\mathcal{A}_{N,q_0}$ with depth two, embedding dimension $3 + m(4 + 2D)$ and $1 + mN$ heads. Its feedforward depth is one and its width is $m(N + 1)$. The parameter count is $O_{D,m}(N)$ and the parameter magnitude is $O(\|q_0\|_\infty + N)$.

For $m = 1$ and $X \subseteq [-M, M]^D$, $M \geq 1$, the same two layers and $N + 1$ heads compute the joint map

$$(f_1, \ldots, f_N) \longmapsto (q_i, q_{i-1})_{i=1}^N, \qquad q_i = f_i(q_{i-1}),$$

exactly. The embedding dimension is $O(D)$, feedforward depth one, feedforward width $O(N + D)$, parameter count $O_D(N)$ and parameter magnitude $O(M + N^2)$. With reset indicators fixed, errors in the supplied values within this cube are not amplified.

Error preservation assumes a fixed initial value and identical reset indicators. *[Lean references: exact resets and error preservation.]*

Appendix B. Triangular products

Let $H \leq \mathrm{GL}_r(\mathbb{C})$ consist of upper-triangular matrices, let $\mathscr{A} \subseteq H$ be finite and nonempty, and fix $g_0 \in H$. Restrict the action $(H, \mathcal{G}(H))$ to the transformations R_A, K_A with $A \in \mathscr{A}$. Choose an integer $B \geq 1$ such that

$$\|g_0\|_\infty,\ \|A\|_\infty,\ |A_{bb}|^{-1} \leq 2^B \qquad (A \in \mathscr{A},\ 1 \leq b \leq r),$$

and put $\Omega = \{(a, b) : a < b,\ A_{ab} \neq 0 \text{ for some } A \in \mathscr{A}\}$. Represent R_A by $(0, 2^{-B}A)$ and K_A by $(1, 2^{-B}A)$, using real and imaginary parts for complex entries. Perturbations lie in $\{0, 1\} \times \mathrm{Mat}_r(\mathbb{C})$ and are measured in the maximum norm of these real coordinates; the first coordinate remains exact. Write Y_i for the successive group elements, with $Y_0 = g_0$.

Lemma B.1 (Simulation of triangular groups)**.** *For every $N \geq 1$ and $0 < \varepsilon < 1$, there are powers of two $\sigma \in (0, 1)$ and $\nu \in (0, 1/2)$ such that the map from a word of length N in the numerical coordinates above to $((\sigma Y_i, \sigma Y_{i-1}))_{i=1}^N$ admits a transformer $\sigma\varepsilon$-approximation at tolerance ν. Its depth is $4 + \tau(\Omega)$, its embedding dimension is $O(r^3)$, its feedforward depth is at most four, and it uses $O_r(N + R^{10r+1})$ heads, where*

$$R = N(B + 1) + \log(|\mathscr{A}| + 2) + \log(1/\varepsilon) + 2.$$

Moreover, $\log(1/\sigma), \log(1/\nu) = O_r(R)$. *There is* $\delta_0 \in (0,1)$ *such that the same parameters and tolerance work for every common normalization stabilizer* $0 < \delta \le \delta_0$. *The remaining complexity measures and* $\log(1/\delta_0)$ *are polynomial in* R.

Notice that the family $\mathscr{A}$ may depend on the horizon and have exponentially many elements; only its logarithmic size enters the bound.

[Lean reference: triangular group simulation.]

For several groups with resets sharing the same reset indices, the preparation layers can be shared and the iterations performed in parallel. The required number of iterations is the largest of their column heights:

Corollary B.2 (Simultaneous triangular products)**.** *For* $q \ge 1$ *instances of Lemma B.1 of size* r, *with common* B *and reset indices, the joint map of current and preceding group elements admits a transformer* $\sigma\varepsilon$*-approximation at tolerance* ν, *with a common scale* σ, *depth* $4 + \max_\kappa \tau(\Omega_\kappa)$, *embedding dimension* $O(qr^3)$, *feedforward depth at most four and* $O_{q,r}(N + R_q^{10qr+1})$ *heads, where*

$$R_q = N(B+1) + \sum_{\kappa=1}^{q} \log(|\mathscr{A}_\kappa| + 2) + \log(1/\varepsilon) + 2.$$

The scale, tolerance and common normalization stabilizer satisfy the same bounds with R_q *in place of* R *and constants depending on* q, r.

[Lean reference: simultaneous triangular products.]

B.1. **Formal source.** *[Formal source, version and reproduction reference to be added.]*

Appendix C. Finite-horizon specialization

Fix a minimal GHMM $\mathcal{P}$ with solvable eventual-image stabilizers and the data chosen in Propositions 4.15 and 4.19 and theorem 4.20. Let $\theta \in \mathbb{R}^{s_\mathcal{P}}$ collect the process parameters, recurrent representatives, witnesses, bases, coset representatives and coefficients of the coset equations. Thus $s_\mathcal{P}$ depends only on $\mathcal{P}$. Replacing θ means evaluating the same formulas with new values for these constants. Use the triangular supports over all original words and put $\omega = 2 + \max_{r,\kappa} \tau(\Omega_{r,\kappa})$, with empty maximum zero.

Proposition C.1 (Finite-horizon specialization)**.** *For every* $N \ge 1$ *and integer* $Q \ge 2$, *there are a parameter tuple* θ_* *in a fixed bounded box and a bound*

$$B = O_\mathcal{P}\Big((N+1)^{\omega(2s_\mathcal{P}+3)}\big((N+1)^\omega \log(N+2) + \log Q\big)\Big)$$

such that, on every word of length at most N*:*

(i) *the cascade makes the same discrete choices, and its chart, group-decomposition and recovery identities remain valid;*

(ii) *all coordinates in its rational calculations are bounded by* 2^B*; every nonzero coordinate, denominator and decision test has magnitude at least* 2^{-B}*;*

(iii) $p_*(w) = \mu_* T_{*,w} \ell_*$ *has the same sign as* $p(w)$, *and*

$$\left\| \frac{\mu_* T_{*,w}}{p_*(w)} - b_w \right\|_\infty \le Q^{-1} \qquad \text{when } p(w) > 0.$$

The same tuple θ_* *is used for all these words.*

[Lean references: common specialization and quantitative bounds.]

Appendix D. Rank computations and belief recovery

D.1. **Implementation of one rank.** Fix $1 \le r \le d$ and one specialization from Proposition C.1 for horizon N. A superscript $*$ denotes evaluation at this specialization. Write $y^*_{r,i}$ for the rank state after position i, with $y^*_{r,0} = \varnothing$, and $E^*_{r,i}$ for its input in (4.3). Enlarge B to cover the fixed changes of basis and their inverses, and put $s = 2^{-B-3}$. Represent a matrix M by $(1, sM)$ and $\varnothing$ by $(0, 0)$. Errors are measured in the maximum norm of these coordinates, with the empty indicators held exact.

Use q_r, λ_r, τ_r as defined after Theorem 3.1, and put $\iota_r = \mathbf{1}_{\{K_r > 1\}}$, where K_r is the block bound in Proposition 4.15. Define

$$e_r = (K_r - 1)d^2 + 2d,$$

$$\gamma_r = \begin{cases} 0, & X_r = \{\varnothing\}, \\ \max\{3d^2 + 1, e_r\}, & X_r \neq \{\varnothing\},\ q_r = 0, \\ \max\{3d^2 + 1, e_r, 10rq_r + 1, 2d^2 + d + 2r^2\}, & q_r > 0. \end{cases} \tag{D.1}$$

Proposition D.2 (Implementation of one rank)**.** *For* $0 < \eta < 1$, *the map on incoming sequences from original words of length at most* N,

$$(E^*_{r,i})_i \longmapsto (y^*_{r,i}, y^*_{r,i-1}, E^*_{r-1,i})_i,$$

admits a transformer $s\eta$*-approximation at tolerance* $\nu > 0$ *in the coordinates above. Empty indicators are exact at input and output. Its depth is at most*

$$L_r = \begin{cases} 0, & X_r = \{\varnothing\}, \\ 8 + 2\iota_r, & X_r \neq \{\varnothing\},\ q_r = 0, \\ 14 + 2\iota_r + \lambda_r + \tau_r, & q_r > 0. \end{cases}$$

Its feedforward depth is at most four, its embedding dimension depends only on $\mathcal{P}$, *and its head count is* $O_{\mathcal{P}}(N + R_r^{\gamma_r})$, *where* $R_r = N(B + 1) + \log(1/\eta) + 2$. *The tolerance satisfies* $\log(1/\nu) = O_{\mathcal{P}}(R_r)$. *The current states of higher ranks may be retained with the same approximation and resource bounds.*

There is $\delta_r \in (0, 1)$ *such that the same parameters and tolerance work for every common normalization stabilizer* $0 < \delta \le \delta_r$. *The remaining complexity measures and* $\log(1/\delta_r)$ *are polynomial in* R_r.

[Lean references: rank simulation, retained states and connections between ranks.]

D.2. **Composition and resources.** Use $s_{\mathcal{P}}$ and ω from Proposition C.1, and set

$$e_{\mathrm{out}} = \begin{cases} 1, & d = 1, \\ (d-2)d^2 + 3d, & d \geq 2. \end{cases}$$

The exponent in Theorem 3.1 is

$$\Gamma = \max\{e_{\mathrm{out}}, \gamma_1, \ldots, \gamma_d\}, \qquad \beta = 2(s_{\mathcal{P}} + 2)\omega, \qquad c_{\mathcal{P}} = (\beta + 1)\Gamma. \tag{D.3}$$

Proposition D.4 (Composition and belief recovery)**.** *For every $N \geq 1$ and $0 < \varepsilon \leq 1$, choose $Q = \lceil 4/\varepsilon \rceil$ and one specialization from Proposition C.1, with bound B. Put $R = N(B+1)+\log(1/\varepsilon)+2$. The rank simulations, rank-zero calculation and final belief recovery can be composed into a transformer that ε-simulates the original belief semiautomaton up to horizon N, with exact support and sink output. Its inputs are the original letters and BOS only. Its depth, feedforward depth and embedding dimension satisfy*

$$L \leq 2 + \sum_{r=1}^{d} L_r \leq 10d + \sum_{r:q_r>0} (6 + \lambda_r + \tau_r), \qquad K \leq 4, \qquad m = O_{\mathcal{P}}(1).$$

Its head count is $O_{\mathcal{P}}(N + R^{\Gamma})$. There is $\delta_0 \in (0,1)$ such that the same parameters give these guarantees for every common normalization stabilizer $0 < \delta \leq \delta_0$. The remaining complexity measures and $\log(1/\delta_0)$ are polynomial in R. For $T = N + \log(1/\varepsilon) + 2$, the specialization bound gives $R = \widetilde{O}_{\mathcal{P}}(T^{\beta+1})$, and hence the head count is $\widetilde{O}_{\mathcal{P}}(T^{c_{\mathcal{P}}})$.

[Lean references: composition, original-belief recovery and resource bounds.]

References

[1] A. S. Shai, S. E. Marzen, L. Teixeira, A. Gietelink Oldenziel and P. M. Riechers, *Transformers Represent Belief State Geometry in their Residual Stream*, NeurIPS 2024. arXiv:2405.15943.

[2] B. Liu, J. T. Ash, S. Goel, A. Krishnamurthy and C. Zhang, *Transformers Learn Shortcuts to Automata*, ICLR 2023. arXiv:2210.10749.

[3] S. Basu, *Algorithms in Real Algebraic Geometry: A Survey*. Author's text.

[4] R. Xiong et al., *On Layer Normalization in the Transformer Architecture*, Proceedings of the 37th International Conference on Machine Learning, PMLR 119 (2020), 10524–10533. Publisher's text.

[5] J. S. Milne, *Algebraic Groups*, 2022. Author's text.

[6] E. Hrushovski, J. Ouaknine, A. Pouly and J. Worrell, *Polynomial Invariants for Affine Programs*, LICS 2018. arXiv:1802.01810.

[7] B. Steinberg, *A Theory of Transformation Monoids: Combinatorics and Representation Theory*, Electronic Journal of Combinatorics 17 (2010), Research Paper R164. Publisher's text.

[8] J.-É. Pin, *Mathematical Grounds of Automata Theory*, lecture notes, version of 14 January 2008. 2008 draft.

[9] M. Krasner and L. Kaloujnine, *Produit complet des groupes de permutations et problème d'extension de groupes. II*, Acta Scientiarum Mathematicarum (Szeged) 14 (1951–1952), 39–66. Journal archive.